

机器学习

Introduction

机器学习的定义：机器学习是一个 **不通过显著式编程** 来赋予计算机学习能力的领域。

- 显著式编程：需要人为地根据周围的环境、规则、经验等给计算机规定一些机械化步骤或判断依据。例如教计算机认识树需要输入能定义树的所有规则
- 非显著式编程：无需人为给出所有的步骤和约束计算机必须总结什么规律

一个计算机程序被称为可学习，是指它能够针对某个**任务T**和某个**性能指标P**，从**经验E**中学习，并且它在T上被P所衡量的性能指标，会随着经验E的增加而提高。

任务T：编写计算机程序识别树的图片

经验E：大量的关于树的图片

性能P：不同算法有不同的指标，如识别的正确率

并且，随着**经验E**（输入树的图片）的增加，**性能P**（识别率）也会提高

能使用机器学习的三个**必要条件**

1. 存在一些**可被学习的潜在模式**，故性能指标P能被提升
2. 不容易被显著式编程
3. 存在关于潜在模式的**数据**，故机器学习有经验E可供输入

下面哪个任务最适合采用机器学习来解决？

- A、预测一个婴儿下一次哭泣的时长是多久
- B、确定一个网络中是否存在环
- C、预测接下来几十年地球会不会因为核能的乱用而毁灭
- ☒ D、银行决定是否要批准某顾客信用卡的申请

机器学习的 **基本术语** （以银行批准一用户的信用卡申请是好/坏为例）

- 年龄，性别，年收入，工作年限被称为属性或特征（feature）
- 特征向量 $x_i = (x_1, x_2, x_3, x_4)^T$ 为输入，每一行被称为一个训练样本（training sample），简称为样本（sample）
- “是否批准”这个信息表示输出，被称为样本的标注（Label）
- **输入：** $x_i \in \mathcal{X}$ ， $x_i = (x_1, x_2, \dots, x_d)^T$ 表示用户的申请示例， x_j 表示属性（特征）， $\mathcal{X}$ 表示样本空间
- **输出：** $y \in \mathcal{Y}$ ， y 表示发卡给该用户好还是坏， $\mathcal{Y}$ 是所有标注的集合
- 目标函数（待被学习的未知模式）： $f: \mathcal{X} \rightarrow \mathcal{Y}$ （理想的）
- 数据：**训练数据集** $\mathcal{D} = \{x_1, y_1, x_2, y_2, \dots, x_m, y_m\}$ 表示历史数据
- 假设/技能（学习到的函数）： $g: \mathcal{X} \rightarrow \mathcal{Y}$ （期望有好的性能）
- **假设集合** $\mathcal{H}$ ：包含好的和不好的假设，算法 $\mathcal{A}$ 从中选择“最好”的作为 g

下面哪个可正确表示电影推荐里面的学习问题？

$S1=[0,100]$

$S2=$ 所有可能的 (用户id, 电影id) 对

$S3=$ 所有可能影响用户和电影关系的公式

$S4=1,000,000$ 对 ((用户id, 电影id) , 评分) 数据

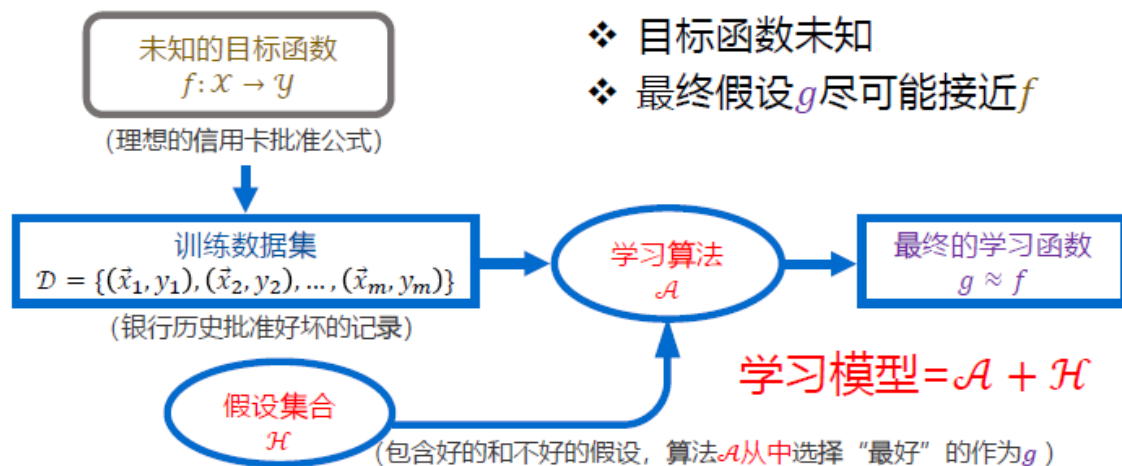
A. $S1=x, S2=y, S3=\mathcal{H}, S4=\mathcal{D}$

☒ B. $S1=y, S2=x, S3=\mathcal{H}, S4=\mathcal{D}$

C. $S1=\mathcal{D}, S2=\mathcal{H}, S3=y, S4=x$

D. $S1=x, S2=\mathcal{D}, S3=y, S4=\mathcal{H}$

➤ 机器学习的工作流程：



分类方式

- 根据训练数据**是否有标注及标注是否全面**，机器学习问题大致划分为：
 - **监督学习**(Supervised Learning)：可分为“分类”和“回归”两类
 - 分类问题：标注为离散值，如是否批准信用卡申请
 - 回归问题：标注为连续值，如预测武汉市房价
 - **无监督学习**(Unsupervised Learning)：常见的“聚类”问题
 - **半监督学习**(Semi-supervised Learning)：只有部分数据有标签
 - 此外，机器学习还有其他类别，**强化学习** (reinforcement-learning)
- 根据**数据的获得和使用方式**，机器学习问题大致划分为：
 - **批量学习**(Batch Learning)：一次性拿到整个数据集 $\mathcal{D}$ ，对其进行学习建模，得到最终的机器学习模型
 - **在线学习**(Online Learning)：数据是实时更新的，根据数据一个个进来，同步更新算法，如在线邮件过滤系统，这是一个动态过程
 - **主动学习**(Active Learning)：近些年来新出现的一种机器学习类型，即让机器具备主动提出标注请求的能力，帮助筛选出重要的label

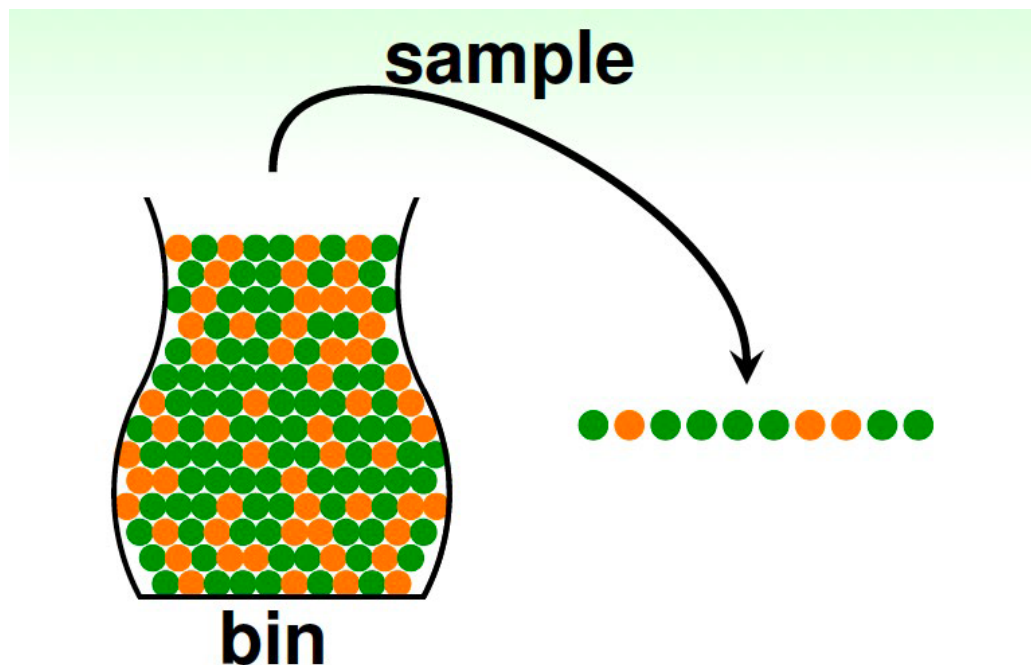
机器学习理论

机器可学习性分析

统计学上如何估计未知量：如何估计罐子里橙色球的比例？

未知量：罐子里橙色球比例为 u

已知量：采样样本中橙色球比例为 v



霍夫丁不等式 (Hoeffding's Inequality)

$$P(|u - v| > \epsilon) \leq 2\exp(-2\epsilon^2 N) \quad (1)$$

当抽样样本(N)足够大时， u 与 v 可能近似相等（在 ϵ 范围内）

这时称“ $u \approx v$ ”是可能近似正确的（probably approximately correct (PAC)）

机器学习要求样本满足**独立同分布**

测验： $u = 0.4$ ，采样 10 个样本得到的比例 $v < 0.1$ ，那 u 和 v 近似相等可能性的上限是多少？

A、0.67 B、0.4 C、0.33 D、0.05

➤ 抽样模型及霍夫丁不等式与学习的联系

罐子抽样模型

- 需确定的量：橙色球比例 u
- 球的空间：罐子
- 橙色球
- 绿色球
- 从罐子里抽 N 个样本



- $h(x) \neq f(x)$
- $h(x) = f(x)$

学习模型

- 需确定的量：给定假设上 $h(\vec{x}) = f(\vec{x})$ 是否成立？
- 输入空间： $\vec{x} \in \mathcal{X}$
- h 正确： $h(\vec{x}) = f(\vec{x})$
- h 错误： $h(\vec{x}) \neq f(\vec{x})$
- 数据集 $\mathcal{D} = \{(x_n, y_n)\}$ 上确定 h 是否正确

数据集规模足够大时，可通过数据集上 $h(\vec{x}) \neq f(\vec{x})$ 的概率来推测输入空间上 $h(\vec{x}) \neq f(\vec{x})$ 的概率

➤ 抽样模型及霍夫丁不等式与学习的联系

定义数据集上的错误率及 $h(\vec{x}) \neq f(\vec{x})$ 的概率为 $E_{in}(h)$ ，样本空间上的错误率为 $E_{out}(h)$ ，当数据集规模很大时，两者在 ε 误差范围内近似相等：

$$P(|E_{in}(h) - E_{out}(h)| > \varepsilon) \leq 2\exp(-2\varepsilon^2 N)$$

- ✓ 若 $E_{in}(h) \approx E_{out}(h)$ 且 $E_{in}(h)$ 很小，
则样本空间上的错误率 $E_{out}(h)$ 小，即 $h \approx f$
- ✓ 若选择假设 h 作为学习函数 g ，且 $E_{in}(h)$ 小，则“ $g \approx f$ ” PAC
- ✓ 反之，则“ $g \neq f$ ” PAC

➤ 测验：根据霍夫定不等式

$$P(|E_{in}(h) - E_{out}(h)| > \varepsilon) \leq 2M\exp(-2\varepsilon^2 N) = \delta$$

可由给定的容忍误差 ε 和对坏样本的容忍误差 δ 来确定需要收集多大的数据集 N 来满足该要求。给定 $\varepsilon = 0.1$, $\delta = 0.05$, $M = 100$,那么需要多大的数据集。

A、 215 B、 415 C、 615 D、 815



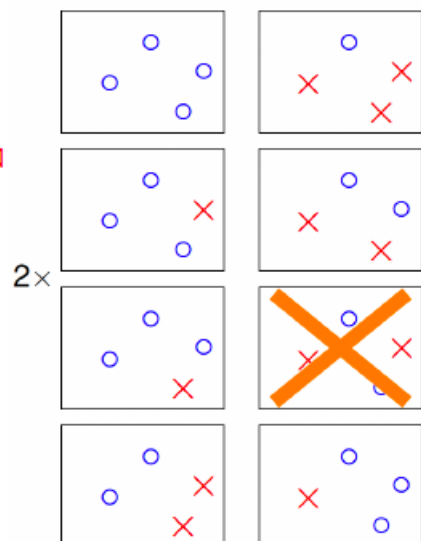
$$N = \frac{1}{2\varepsilon^2} \ln \frac{2M}{\delta}$$

2、Hoeffding不等式与模式二分性

✓ 下面考虑4个样本的二分类模型

该模型无限多的假设可被归类为右边14种
即有效直线数量为 $effective(N) = 14$

➤ 4个样本的感知器分类模型存在
 $14 < 2^3$ 种dichotomies



- 用成长函数 $m_{\mathcal{H}}(N)$ 代替 M ，霍夫定不等式可写成

$$P(|E_{in}(h) - E_{out}(h)| > \varepsilon) \leq 2m_{\mathcal{H}}(N) \exp(-2\varepsilon^2 N)$$

- ✓ 若 $m_{\mathcal{H}}(N)$ 为多项式，则

$$E_{in}(h) \approx E_{out}(h) \text{ 成立}$$

- ✓ 若 $m_{\mathcal{H}}(N)$ 为指数函数，则

$$E_{in}(h) \approx E_{out}(h) \text{ 不成立}$$

- 2维平面上感知器学习模型的成长函数 $m_{\mathcal{H}}(N)$ 为：

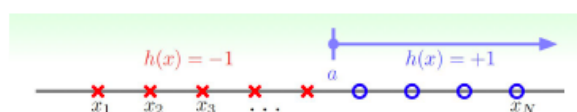
N	$m_{\mathcal{H}}(N)$
1	2
2	4
3	$\max(\dots, 6, 8) = 8$
4	$\max(\dots, 8, 14) = 14 < 2^4$
...	...
N	$< 2^N$

- 1维平面上感知器学习模型的成长函数 $m_{\mathcal{H}}(N)$

❖ Positive Rays

$$h(x) = \text{sign}(x - a)$$

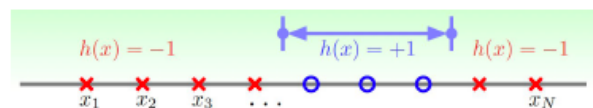
$$m_{\mathcal{H}}(N) = N + 1$$



❖ Positive Intervals

$$h(x) = \begin{cases} +1, & \text{if } x \in [l, r] \\ -1, & \text{其他} \end{cases}$$

$$m_{\mathcal{H}}(N) = C_{N+1}^2 + 1 = \frac{1}{2}N^2 + \frac{1}{2}N + 1$$



- 不同假设空间对应的成长函数 $m_{\mathcal{H}}(N)$

- ❖ 一维空间上 positive rays

$$m_{\mathcal{H}}(N) = N + 1$$

- ❖ 一维空间上 positive intervals

$$m_{\mathcal{H}}(N) = \frac{1}{2}N^2 + \frac{1}{2}N + 1$$

- ❖ Convex set

$$m_{\mathcal{H}}(N) = 2^N$$

- ❖ 二维平面上的感知器模型

$$m_{\mathcal{H}}(N) \leq 2^N$$

- 若 k 个样本的任意输入都不能被 $\mathcal{H}$ 所打散，即 $m_{\mathcal{H}}(k) < 2^k$ 则称 k 为 $\mathcal{H}$ 的 **break point (突破点)**，且 $k+1, k+2, \dots$ 也是 break point。我们关注最小的 break point

定理： 若存在 break point $k \geq 3$ ，且 $N \geq 2$ ，则 $m_{\mathcal{H}}(N) \leq N^{k-1}$

证明可参考：**Learning from data**, 2012, Abu-Mostafa et al.

➡ 若对于 $\mathcal{H}$ 存在 break point k ，则 $E_{in}(g) \approx E_{out}(g)$

测验：直线排列的二分类模型中考虑假设空间 $\mathcal{H}$ 为positive and negative rays, 即 $h(x) = \pm \text{sign}(x - a)$, 其成长函数 $m_{\mathcal{H}}(N) = ?$,

- A、 N B、 $N + 1$ C、 $2N$ D、 2^N

$\mathcal{H}$ 最小的break point是多少?

- A、1 B、2 C、3 D、4



如3样本情形

VC 维

➤ VC维 (VC Dimension) 的定义

假设空间 $\mathcal{H}$ 的VC维 $d_{VC}(\mathcal{H})$ 表示 $\mathcal{H}$ 所能“打散” (Shatter) 的最大样本数 N_s , 即 ($d_{VC} = N_s$)
 N_s 是使得 $m_{\mathcal{H}}(N_s) = 2^{N_s}$ 成立的最大值

➤ $d_{VC}(\mathcal{H}) = \text{最小的break point } k - 1$

➤ 若 $N_s \geq 2$, 即 $d_{VC} \geq 2$, 则 $m_{\mathcal{H}}(N) \leq N^{d_{VC}}$

➤ 不同假设空间对应的VC维

❖ positive rays

❖ positive intervals

❖ Convex set

❖ 二维平面上的感知器模型

$$m_{\mathcal{H}}(N) = N + 1$$

$$m_{\mathcal{H}}(N) = \frac{1}{2}N^2 + \frac{1}{2}N + 1$$

$$m_{\mathcal{H}}(N) = 2^N$$

$$m_{\mathcal{H}}(N) \leq N^3, \text{对于 } N \geq 2$$

$$d_{VC} = 1$$

$$d_{VC} = 2$$

$$d_{VC} = \infty$$

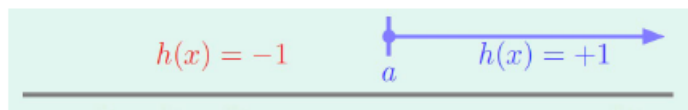
$$d_{VC} = 3$$

$$P(|E_{in}(g) - E_{out}(g)| > \varepsilon) \leq 4(2N)^{d_{VC}} \exp\left(-\frac{1}{8}\varepsilon^2 N\right)$$

➤ 若VC维 d_{VC} 有限, 则在样本数 N 大的情况下: $E_{in}(g) \approx E_{out}(g)$

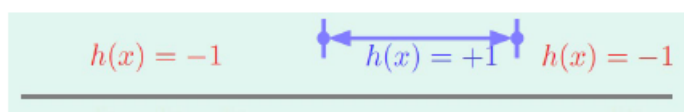
➤ VC维可能的物理意义

❖ positive rays



$$d_{VC} = 1 \quad h(x) = \text{sign}(x - a) \quad \text{自由参数: } a$$

❖ positive intervals



$$d_{VC} = 2 \quad h(x) = \begin{cases} +1, & \text{if } x \in [l, r] \\ -1, & \text{其他} \end{cases} \quad \text{自由参数: } l, r$$

VC维 $d_{VC} \approx$ 自由参数的数量 (并不总是成立)

➤ 测验: 若 $d_{VC} \leq d + 1$, 下面哪个说法是正确的?

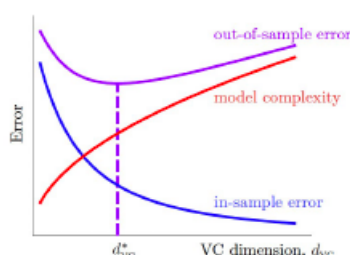
- A、存在某个或某些 $d + 1$ 数量的输入能被 “打散”
- B、任何数量为 $d + 1$ 的输入都能被 “打散”
- C、存在某个或某些 $d + 2$ 数量的输入不能被 “打散”
- ✓ D、任何数量为 $d + 2$ 的输入都不能被 “打散”

3、VC维理论与模型复杂性

➤ 式子 $\Omega(N, \mathcal{H}, \delta) = \sqrt{\frac{8}{N} \ln\left(\frac{4(2N)^{d_{VC}}}{\delta}\right)}$ 表示 **模型复杂度**

➤ 预测误差 $E_{out}(g) \leq E_{in}(g) + \sqrt{\frac{8}{N} \ln\left(\frac{4(2N)^{d_{VC}}}{\delta}\right)}$

➤ 误差 $E_{out}(g)$ 和 $E_{in}(g)$, **模型复杂度 Ω** 与 **VC维** 的关系如下:



- $d_{VC} \uparrow$: $\Omega \uparrow$, $E_{in} \downarrow$
- $d_{VC} \downarrow$: $\Omega \downarrow$, $E_{in} \uparrow$
- 最好的 d_{VC}^* 是中间某个值

模型优化与验证方法

模型选择方法: 留出法、交叉验证法、自助法

留出法

直接将数据集 D 划分为两个互斥的集合，即 $D = S \cup T$ ，且 $S \cap T = \emptyset$

在 S 上训练出模型后，用 T 来评估其测试误差，作为对泛化误差的估计

- 存在的矛盾： S 和 T 大小的分配问题
 - S 大 T 小，评估结果不够稳定准确；
 - S 小 T 大，训练出的模型与用 D 训练出的模型可能有较大差别

➤ **测验：**给定包含1000个样本的数据集，其中500个正例，500个反例，将其划分为包含70%样本的训练集和30%样本的测试集用于留出法评估，试计算有多少种划分方式。

$$C_{500}^{300} \times C_{500}^{300}$$

更正为350

交叉验证法

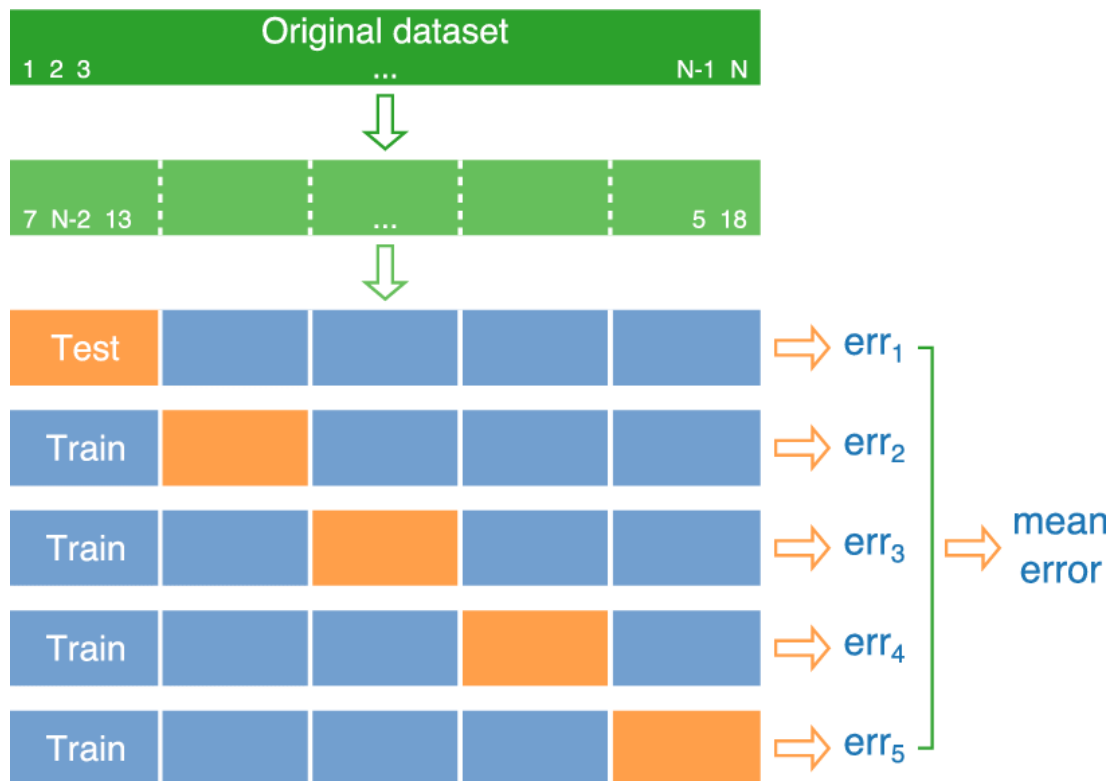
将数据集 D 划分为分布尽可能一致 k 个互斥子集，即 $D = D_1 \cup D_2 \dots \cup D_k$ ，且 $D_i \cap D_j = \emptyset \ i \neq j$ ；

每次用 $k-1$ 个子集的并集作为训练集，余下的子集作为测试集；

返回这 k 个测试结果的均值

交叉验证法（cross validation），也称为“ k 折交叉验证”

随机使用不同的划分重复 p 次取均值



自助法

1. 给定 m 个样本的数据集 D ，有放回地采样得到包含 m 个样本的数据集 D'
2. 用 D' 作为训练集， $D \setminus D'$ 用作测试集
 - ✓ 有部分样本在 D' 中重复出现，而另一部分样本不出现
始终不出现的概率为 $(1 - \frac{1}{m})^m$ ，其极限 $\lim_{m \rightarrow \infty} (1 - \frac{1}{m})^m = \frac{1}{e} \approx 0.368$
 - ✓ 在数据集小，难以划分训练集和测试集有用
 - ✓ 改变了初始数据集的分布，引入估计偏差，数据量足够时，留出法和交叉验证法更常用

模型性能度量指标

➤ 模型性能度量

模型泛化能力的评价标准，可使用 E_{out} 来度量， E_{out} 可通过 E_{in} 来估计。
给定 m 个样本的数据集 D ，样本的真实标记为 y_i ，模型预测的结果为 $g(x_i)$

- 对于回归任务，其性能可用均方误差 $E(g; D)$ 来度量

$$E(g; D) = E_{in} = \frac{1}{m} \sum_{i=1}^m (g(x_i) - y_i)^2$$

- 对于分类任务，其性能可用错误率 $E(g; D)$ 来度量

$$E(g; D) = E_{in} = \frac{1}{m} \sum_{i=1}^m \mathbb{I}(g(x_i) \neq y_i)$$

或用精度 $acc(g; D)$ 来度量 $acc(g; D) = 1 - E(g; D)$

➤ 模型性能度量

对分类任务，除常用的错误率与精度外，还有如下性能度量指标

1. 查准率 (Precision)、查全率 (Recall) 与F1

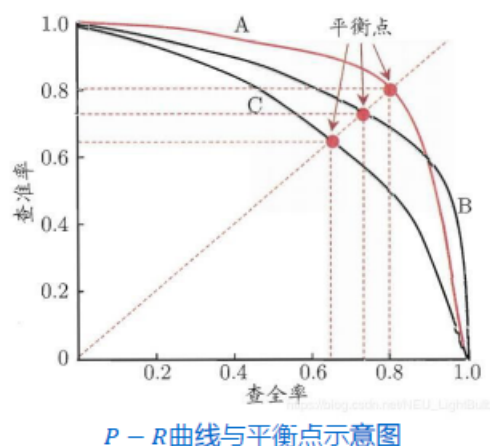
真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

分类结果混淆矩阵

查准率 P 度量被预测为正样本的结果中有多少真正的正样本 $P = \frac{TP}{TP + FP}$

查全率 R 度量所有真实的正样本中有多少被预测正确 $R = \frac{TP}{TP + FN}$

1. 查准率 (Precision)、查全率 (Recall) 与 F1



P-R曲线与平衡点示意图

- ✓ 查准率 P 与查全率 R 间存在矛盾
- ✓ 可采用 $P-R$ 曲线下面积度量模型性能，缺点：不太容易估算
- ✓ 可采用“平衡点”作为模型性能度量，缺点：过于简化
- 常用两者的调和平均 $F1$ 度量

$$\frac{1}{F1} = \frac{1}{2} \left(\frac{1}{P} + \frac{1}{R} \right) \rightarrow F1 = \frac{2 * P * R}{P + R}$$

2. ROC与AUC

- ✓ 很多模型为样本产生一个预测值，若该预测值大于一个选定的分类阈值，则判为正例，否则为反例
- ✓ 根据预测值的大小可对样本进行排序，针对不同任务可采用不同阈值来截断，而该排序体现了模型在一般情况下的泛化性能的好坏
- ✓ **ROC** (Receiver Operating Characteristic, 受试者工作特征) 曲线可用于度量该模型的泛化性能

ROC曲线的纵轴是“真正例率” (True Positive Rate, TPR)

$$TPR = \frac{TP}{TP + FN}$$

ROC曲线的横轴是“假正例率” (False Positive Rate, FPR)

$$FPR = \frac{FP}{TN + FP}$$

3. 代价敏感错误率

现实任务的不同错误导致的后果不同，会产生不同的代价

真实类别	预测类别	
	正例	反例
正例	0	$cost_{01}$
反例	$cost_{10}$	0

二分类代价矩阵

此时，对应的“代价敏感”错误率为

$$E(g; \mathcal{D}; cost) = \frac{1}{m} \left(\sum_{x_i \in \mathcal{D}^+} \mathbb{I}(g(x_i) \neq y_i) \times cost_{01} + \sum_{x_i \in \mathcal{D}^-} \mathbb{I}(g(x_i) \neq y_i) \times cost_{10} \right)$$

- **测验：** 在一个指纹门禁系统中，有999, 990个正常用户的正例样本 $y_i = +1$ ，10个入侵者的反例样本 $y_i = -1$ ，若采用的模型 $g(x)$ 总是返回常数+1，那么该模型的代价敏感错误率为多少？

A. 0.001

B. 0.01 ✓

C. 0.1

D. 1

		$g(x)$	
		+1	-1
y	+1	0	1
	-1	1000	0

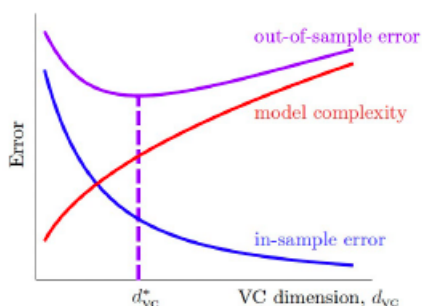
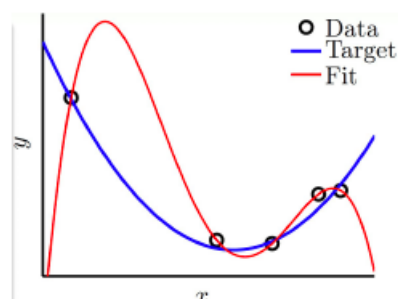
过拟合与正则化

在模型将训练样本学得“太好”的时候，会把训练样本本身特点当成一般性质，从而导致泛化性能下降，这中现象叫做**过拟合** (Overfitting)

如右图给定目标函数为2阶多项式，从中取5个点并加上少量的噪声作为训练样本集 $\mathcal{D}$

✓ 学习到的模型为4阶多项式，穿过这5个数据点 $E_{in} = 0$

✓ 而对应的泛化误差 E_{out} 却很大



✓ 在最优 d_{VC}^* 的右侧，随着VC维的增大， $E_{in} \downarrow$ ， $E_{out} \uparrow$ ，发生**过拟合**现象

✓ 在最优 d_{VC}^* 的左侧，随着VC维的减小， $E_{in} \uparrow$ ， $E_{out} \uparrow$ ，发生**欠拟合** (underfitting) 现象

VC维的大小会影响过拟合现象

目标函数为10次多项式+噪声

	$g_2 \in \mathcal{H}_2$	$g_{10} \in \mathcal{H}_{10}$
E_{in}	0.050	0.034
E_{out}	0.127	9.00

✓ 影响因素：噪声

目标函数为50次多项式、无噪声

	$g_2 \in \mathcal{H}_2$	$g_{10} \in \mathcal{H}_{10}$
E_{in}	0.029	0.00001
E_{out}	0.120	7680

✓ 影响因素：问题复杂度 (维度灾难)

✓ 此外，数据集的规模也是一重要的影响因素

影响过拟合现象的因素：

数据集规模、噪声、问题复杂度、VC维

测验：给定一个数据集，下面那种情形发生过拟合现象的风险最小？

- A. 小噪声，学习函数的VC维从小增大到中等大小 ✓
- B. 小噪声，学习函数的VC维从小增大到很大
- C. 大噪声，学习函数的VC维从小增大到中等大小
- D. 大噪声，学习函数的VC维从小增大到很大

如何减少过拟合现象的产生？

- ✓ 从简单的学习函数开始进行拟合
- ✓ 对训练数据集里label明显错误的样本进行修正或删除 (data cleaning/pruning)
- ✓ 对样本数少的情形，可对已知样本进行简单处理、变换，从而获得更多的样本 (Data hinting)
- ✓ 正则化 (Regularization) 等

正则化

正则化思想：从高次函数假设空间 $\mathcal{H}_{10}$ 退回低次函数空间 $\mathcal{H}_2$

如何从高次函数假设空间 $\mathcal{H}_{10}$ 退回低次函数空间 $\mathcal{H}_2$ ？

假设空间 $\mathcal{H}_{10}$ 的函数： $h(x) = w_0 + w_1x + w_2x^2 + \dots + w_{10}x^{10}$

假设空间 $\mathcal{H}_2$ 的函数： $h(x) = w_0 + w_1x + w_2x^2$

- 当权重满足 $w_3 = w_4 = \dots = w_{10} = 0$ 时， $\mathcal{H}_{10} = \mathcal{H}_2$
- 定义权重向量 $\mathbf{w}^T = [w_0, w_1, \dots, w_n]$ ，即 $\mathbf{w} \in \mathbb{R}^{n+1}$

假设空间 $\mathcal{H}_{10}$ 拟合：

$$\min_{\mathbf{w} \in \mathbb{R}^{10+1}} E_{\text{in}}(\mathbf{w})$$

假设空间 $\mathcal{H}_2$ 拟合：

$$\begin{aligned} \min_{\mathbf{w} \in \mathbb{R}^{10+1}} E_{\text{in}}(\mathbf{w}) \\ \text{s.t. } w_3 = w_4 = \dots = w_{10} = 0 \end{aligned}$$

推广到更一般的情形：

- 对于向量 $\mathbf{w}$, $\|\mathbf{w}\|_0 = \sum_{q=0}^{10} \mathbb{I}(w_q \neq 0)$ 表示其0-范数
 $\|\mathbf{w}\|_1 = \sum_{q=0}^{10} |w_q|$ 表示其1-范数
 $\|\mathbf{w}\|_2 = \sqrt{\sum_{q=0}^{10} w_q^2}$ 表示其2-范数

➤ 将假设空间 $\mathcal{H}'_2$ 的限制条件进行变换:

假设空间 $\mathcal{H}'_2$ 拟合:

$$\begin{aligned} \min_{\mathbf{w} \in \mathbb{R}^{10+1}} E_{\text{in}}(\mathbf{w}) \\ \text{s.t. } \sum_{q=0}^{10} \mathbb{I}(w_q \neq 0) \leq 3 \end{aligned}$$

假设空间 $\mathcal{H}(C)$ 拟合:

$$\begin{aligned} \min_{\mathbf{w} \in \mathbb{R}^{10+1}} E_{\text{in}}(\mathbf{w}) \\ \text{s.t. } \|\mathbf{w}\|_2^2 = \sum_{q=0}^{10} w_q^2 \leq C \end{aligned}$$

• $\mathcal{H}(0) \subset \mathcal{H}(1.024) \subset \dots \subset \mathcal{H}(1024) \subset \dots \subset \mathcal{H}(\infty) = \mathcal{H}_{10}$

➤ 测验: 对于 $Q \geq 1$, 下面哪个假设权重向量 ($\mathbf{w} \in \mathbb{R}^{Q+1}$) 不属于正则化假设集合 $\mathcal{H}(1)$

- A. $\mathbf{w}^T = [0, 0, \dots, 0]$
 B. $\mathbf{w}^T = [1, 0, \dots, 0]$
 ✓ C. $\mathbf{w}^T = [1, 1, \dots, 1]$
 D. $\mathbf{w}^T = [\sqrt{\frac{1}{Q+1}}, \sqrt{\frac{1}{Q+1}}, \dots, \sqrt{\frac{1}{Q+1}}]$

特征选择方法: 过滤式、包裹式、嵌入式

特征是在观测现象中的独立、可测量的属性

- 对当前学习任务有用的属性称为“相关特征”
- 对当前学习任务没用的属性称为“无关特征”

过滤式选择

✓ 特征选择与后续学习无关

✓ 设计一个相关统计量分量 δ^j 来度量每个特征 j 的重要性, 如Relief法

对样本 x_i , 在其同类样本中选择最近邻样本 $x_{i,nh}$, 称为“猜中近邻”

对样本 x_i , 在其异类样本中选择最近邻样本 $x_{i,nm}$, 称为“猜错近邻”

$$\delta^j = \sum_i [-\text{diff}(x_i^j, x_{i,nh}^j)^2 + \text{diff}(x_i^j, x_{i,nm}^j)^2]$$

x_a^j 表示样本 x_a 在属性 j 上的取值; 若 $x_i^j = x_{i,nh}^j$, 则 $\text{diff}(x_i^j, x_{i,nh}^j) = 0$, 否则为 1 (或 $|x_a^j - x_b^j|$)

✓ 特征子集的重要性则由子集中每个特征的 δ^j 之和决定

■ 包裹式选择

- ✓ 将最终模型的性能作为特征子集的评价标准，如LVW(Las Vegas Wrapper)方法
 1. 初始化最优特征子集 $A^* = A$ ， A 为初始完整特征子集，训练模型，用交叉验证法估计其误差
 2. 随机产生特征子集 A' ，训练模型并计算误差，若它比当前 A^* 对应的误差更小，则令 $A^* = A'$
 3. 重复步骤2，直至运行时间达到上限，输出 A^*

最终模型性能比过滤式选择的更好，但计算开销大，若时间限制，有可能给不出解

■ 嵌入式选择

- ✓ 特征选择过程与模型训练过程融为一体，在模型训练的过程中自动进行特征选择。如 L_1 正则化

用 L_1 范数替代多项式拟合中的 L_2 范数的平方，模型优化函数为

$$\min_{\mathbf{w}} \sum_{i=0}^m (\mathbf{w}^T \mathbf{x}_i - y_i)^2 + \lambda \|\mathbf{w}\|_1 \quad \text{其中} \|\mathbf{w}\|_1 = \sum_q w_q$$

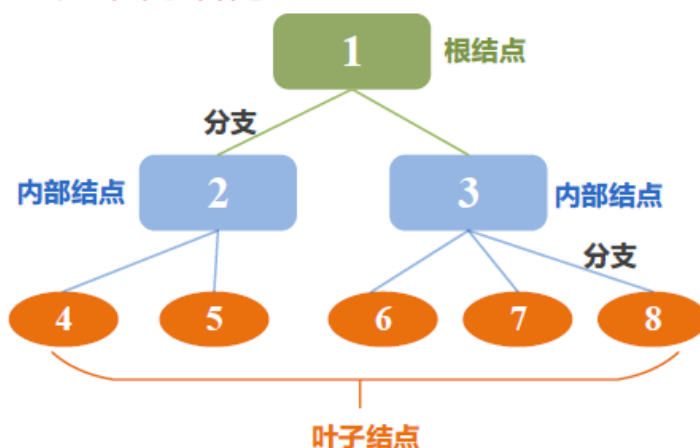
用梯度下降法求使得 $\sum_{i=0}^m (\mathbf{w}^T \mathbf{x}_i - y_i)^2$ 最小的 $\mathbf{w}$ ，同时考虑 L_1 范数的最小化

西瓜书P253

决策树

决策树的结构和构成

➤ 决策树结构



根结点：进行初始属性测试
(该结点包含所有样本)

分支：代表上层结点测试结果的输出

内部结点：进行属性测试(包含部分结点)

叶子结点：表示决策结果

三种典型的决策树算法

- (ID3,C4.5,CART树)的思想, 优缺点 (不需要记信息增益、信息增益比的公式)

ID 3

核心思想: **信息增益**

具体方法:

- 从根结点开始, 计算所有属性的信息增益
- 选择信息增益最大的属性作为结点划分属性
- 由该属性的不同取值进行分支
- 重复以上步骤

优点

- 假设空间包含所有的决策树, 搜索空间完整
- 健壮性好, 不受噪声影响
- 方法简单, 理论清晰

缺点

- 只考虑离散属性, 没有考虑连续属性
- 对缺失值状况没有进行考虑
- 没有考虑过拟合情况
- **采用信息增益作为划分标准**, 但信息增益倾向于取值较多的属性。

ID	年龄	有工作	有自己的房子	信贷情况	类别
1	青年	否	否	一般	否
2	青年	否	否	好	否
3	青年	是	否	好	是
4	青年	是	是	一般	是
5	青年	否	否	一般	否
6	中年	否	否	一般	否
7	中年	否	否	好	否
8	中年	是	是	好	是
9	中年	否	是	非常好	是
10	中年	否	是	非常好	是
11	老年	否	是	非常好	是
12	老年	否	是	好	是
13	老年	是	否	好	是
14	老年	是	否	非常好	是
15	老年	否	否	一般	否

针对前面提到的贷款申请的例子，使用ID3算法来决定是否批准贷款申请。

类别	ID	数量
是	3,4,8,9,10,11,12,13,14	9
否	1,2,5,6,7,15	6

计算根结点信息熵：

$$Ent(D) = - \sum_{k=1}^y p_k \log_2 p_k = - \left(\frac{9}{15} \log_2 \frac{9}{15} + \frac{6}{15} \log_2 \frac{6}{15} \right) = 0.971$$

属性集集合 $A = \{\text{年龄}, \text{工作}, \text{房子}, \text{信贷情况}\}$ ，分别用 A_1, A_2, A_3, A_4 表示集合中的属性元素。

$$H(D|A_1 = \text{青年}) = - \left(\frac{2}{5} \log_2 \frac{2}{5} + \frac{3}{5} \log_2 \frac{3}{5} \right) = 0.971$$

$$H(D|A_1 = \text{中年}) = - \left(\frac{3}{5} \log_2 \frac{3}{5} + \frac{2}{5} \log_2 \frac{2}{5} \right) = 0.971$$

$$H(D|A_1 = \text{老年}) = - \left(\frac{4}{5} \log_2 \frac{4}{5} + \frac{1}{5} \log_2 \frac{1}{5} \right) = 0.722$$

$$H(D|A_1) = \frac{5}{15} * 0.971 + \frac{5}{15} * 0.971 + \frac{5}{15} * 0.722 = 0.888$$

在 A_1 条件下，数据集的经验条件熵

$$Gain(D, A_1) = H(D) - H(D|A_1) = 0.971 - 0.888 = 0.083$$

同理可得

$$Gain(D, A_2) = 0.324$$

$$Gain(D, A_3) = 0.420$$

$$Gain(D, A_4) = 0.363$$

A_3 (即工作)的信息增益最大，因此选择 A_3 为根结点的划分属性，接下来对其他结点递归调用上述方法，建立决策树

➤ 测验：

下面关于ID3算法中说法错误的是（ ）

- A. ID3算法要求特征必须离散化
- B. 采用信息增益作为划分准则
- C. 选取信息增益最大的特征，作为树的根节点
- D. ID3算法是一个二叉树模型

D

C4.5算法

思想

- 连续属性离散化
- 针对缺失值情况给出解决方法
- 引入悲观剪枝进行后剪枝
- 使用**信息增益率**来代替信息增益

➤ 测验：

以下哪个不是C4.5算法的特点？

- A.连续的特征离散化处理
- B.使用信息增益比
- C.通过剪枝算法解决过拟合
- D.对于缺失值的情况没有做考虑

D

CART算法

掌握CART树的划分思路，针对表格型数据，掌握分类问题的基尼指数计算公式以及根节点划分准则（注意要遍历所有属性）

思想

二分递归分割 --> 二叉树

每个属性值都**分割为两部分**

CART决策树采用“基尼指数”来选择划分属性。由于熵模型拥有大量耗时的对数运算，基尼指数在简化模型的同时还保留了熵模型的优点。

数据集D的纯度可以用基尼值来度量：

$$Gini(D) = \sum_{k=1}^y \sum_{k' \neq k} p_k p_{k'} = 1 - \sum_{k=1}^y p_k^2$$

基尼值反映了从数据集D中随机抽取两个样本，其类别标记不一致的概率，因此， $Gini D$ 越小，则数据集D的纯度越高。

计算方法：

ID	年龄A1	有工作A2	有自己的房子A3	信贷情况A4	类别
1	青年	否	否	一般	否
2	青年	否	否	好	否
3	青年	是	否	好	是
4	青年	是	是	一般	是
5	青年	否	否	一般	否
6	中年	否	否	一般	否
7	中年	否	否	好	否
8	中年	是	是	好	是
9	中年	否	是	非常好	是
10	中年	否	是	非常好	是
11	老年	否	是	非常好	是
12	老年	否	是	好	是
13	老年	是	否	好	是
14	老年	是	否	非常好	是
15	老年	否	否	一般	否

数据集D的基尼值定义为

$$Gini(D) = 1 - \sum_{k=1}^y p_k^2 = 1 - \left(\frac{6}{15}\right)^2 - \left(\frac{9}{15}\right)^2 = 0.48$$

当按照是否有房子（二分类）划分：

$$Gini(A3 = \text{是}) = 1 - \left(\frac{6}{6}\right)^2 - \left(\frac{0}{6}\right)^2 = 0$$

$$Gini(A3 = \text{否}) = 1 - \left(\frac{3}{9}\right)^2 - \left(\frac{6}{9}\right)^2 = 0.444$$

$$Gini_index(D, A3) = \left(\frac{6}{15}\right) * Gini(A3 = \text{是}) + \left(\frac{9}{15}\right) * Gini(A3 = \text{否}) = 0.2667$$

ID	年龄A1	类别
1	青年	否
2	青年	否
3	青年	是
4	青年	是
5	青年	否
6	中年	否
7	中年	否
8	中年	是
9	中年	是
10	中年	是
11	老年	是
12	老年	是
13	老年	是
14	老年	是
15	老年	否

当按照年龄A1（三分类）划分：

为保证二分，将数据分成以下三种情况

{青年}, {中年, 老年}; {中年}, {青年, 老年}; {老年}, {青年, 中年}
以{青年}, {中年, 老年}为例计算基尼指数

$$Gini(A1 = \text{青年}) = 1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2 = 0.48$$

$$Gini(A1 = \text{中年, 老年}) = 1 - \left(\frac{7}{10}\right)^2 - \left(\frac{3}{10}\right)^2 = 0.42$$

$$Gini_index(D, A1) = \left(\frac{5}{15}\right) * Gini(A1 = \text{青年}) + \left(\frac{10}{15}\right) * Gini(A1 = \text{中年老年}) = 0.44$$

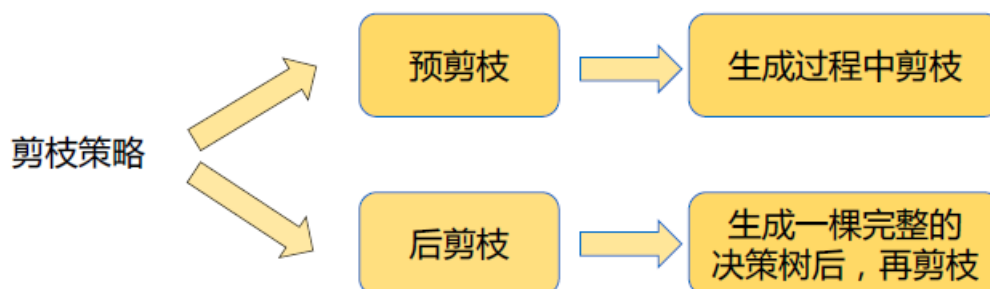
计算其余两种分类情况下的基尼指数，选取基尼指数最小的分类方式，然后在所有属性中寻找基尼指数最小的属性作为根结点，并递归找到其他结点

注意再看一下那三个决策树实例

剪枝策略

预剪枝和后剪枝的对比，从树的生成、泛化性能、训练时间角度理解

剪枝策略是在决策树算法中用来对付“过拟合”的一种方法。



预剪枝

- 在结点划分之前来确定是否继续划分。
- 判断划分是否会使决策树泛化性能提升，若不能，就停止划分。

后剪枝

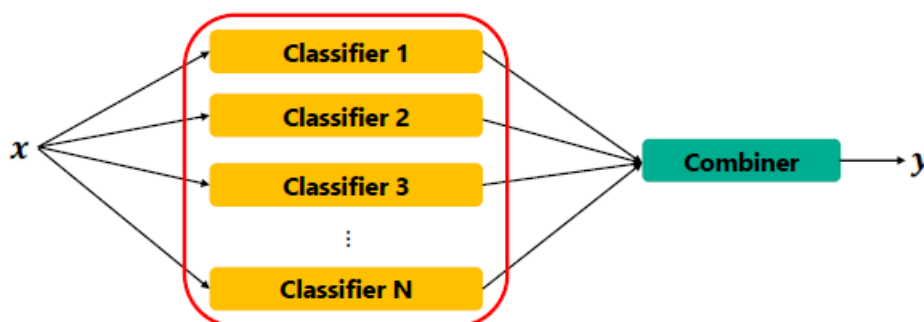
- 在**已经生成的决策树**上进行剪枝
- 自底而上，递归判断将非叶子结点替换为叶结点，**泛化能力是否提升**，若是，则将该节点对应的子树替换为叶结点。
- 相较于预剪枝，后剪枝的**泛化能力更强**，**训练时间更长**

集成学习部分

集成学习的框架

集成学习的框架组成，有哪几部分组成，各部分的作用

- 集成学习的方法是通过训练多个学习器，并通过学习器的合并来解决一个问题。
- 由多个基础学习器(base learner/classifier)构成，而基学习器由基学习算法 (base learning algorithm) 在训练数据上训练获得，可以是前面我们学到的决策树、简单的分类算法如SVM、或者是复杂的神经网络等各种学习方法。
- 集成学习的决策由最终的集成器 (Combiner) 采用一定的集成策略生成，它使得整个集成算法具有更强的泛化能力



Bagging和Boosting的思想

(不需要记伪代码和算法图，不考包外样本)

根据基分类器的生成方式，集成学习可以分为两种：串行生成基分类器的“**串行集成方法**”，以及并行生成分类器的“**并行集成方法**”。

Boosting
(Adaboost, GBDT,
XGBoost)

Bagging
(Random Forest)

- **并行集成方法**：其内在原理是利用基学习器之间的**独立性**，这是因为结合相互独立的基分类器能够显著减小方差 (Variance) 。
- 由于基分类器由相同的训练数据产生，不是完全独立同分布的，因此很难得到完全独立的基分类器，但可以通过**随机的方法 (Bagging 采样方法)** 减少他们之间的相关性，并通过集成实现更好的泛化能力。

以一个二分类任务为例，输出分别为+1和-1，假设真实的函数为 $f(x)$ ，训练 T 个基分类器 $h_i(t)$ ， $i = 1, \dots, T$ 。每个基分类器的判断错误的概率为 ϵ ，对于每一个基分类器，假设弱分类器的分类错误的概率 $\epsilon < \frac{1}{2}$ ，存在：

$$P(h_i(x) \neq f(x)) = \epsilon$$

结合 T 个分类器，根据Voting的方法来决定最后的分类结果 $H(x)$ ，用符号函数表示，可得

$$H(x) = \text{sign}(\sum_{i=1}^T h_i(x))$$

集成后的 H 之后在至少一半的基分类器预测错误时才会犯错。

➤ 测验：Bagging和Boosting有什么不同

1) 样本选择上：

Bagging：训练集是在原始集中有放回选取的，从原始集中选出的各轮训练集之间是独立的。

Boosting：每一轮的训练集不变，只是训练集中每个样例在分类器中的权重发生变化。而权值是根据上一轮的分类结果进行调整。

2) 样例权重：

Bagging：使用均匀取样，每个样例的权重相等

Boosting：根据错误率不断调整样例的权值，错误率越大则权重越大。

3) 预测函数：

Bagging：所有预测函数的权重相等。

Boosting：每个弱分类器都有相应的权重，对于分类误差小的分类器会有更大的权重。

4) 并行计算：

Bagging：各个预测函数可以并行生成。

Boosting：各个预测函数只能顺序生成，因为后一个模型参数需要前一轮模型的结果。

随机森林

- 随机森林(Random Forest, 简称RF) 是Bagging 的一个扩展变体。RF 是在以**决策树为基学习器**构建 Bagging 集成的基础上，**进一步在决策树的训练过程中引入了随机属性选择**。
- 具体来说，传统决策树在选择划分属性时是在当前结点的属性集合(假定有 d 个属性) 中选择一个最优属性；
- 而在 RF 中，对基决策树的每个结点，先从该结点的属性集合中**随机选择一个包含 K 个属性的子集**，然后再从这个子集中选择一个最优属性用于划分。样本选择时采用 Bootstrap 的采样方式。

Bagging和随机森林与决策树方法相比的优势

掌握思想即可，不需要记理论探讨公式

- Bagging可以明显**降低方差**，因此在不稳定学习器上格外有效
- 可以降低 **方差、偏差、泛化误差**

Adaboost、GBDT的特点

以树模型为例，通过什么方式**生成一颗新的树**

这儿没懂

Adaboost 与GBDT特点总结：

- **AdaBoost算法**：加法模型，损失函数为指数函数，学习算法为前向分步算法时的分类问题
- **GBDT算法**：加法模型，学习算法为前向分步算法，**基函数为CART树**，损失函数为平方损失函数的回归问题，为指数函数的分类问题和为一般损失函数的一般决策问题。

针对基学习器的不足：

- AdaBoost算法是通过**提升错分数据点的权重**来定位模型的不足，
- GBDT算法是**通过算梯度**来定位模型的不足。

GBDT 的例子

Gradient Boosting 是提升树 (Boosting Tree) 的一种改进算法
提升树 (Boosting Tree) ：以决策树为基函数/基学习器的提升方法成为**提升树**

输入：训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$;

步骤：

1. 初始化 $f_0(x) = 0$

2. 对 $m = 1, 2, \dots, M$

3. 计算残差 $r_{mi} = y_i - f_{m-1}(x_i), i = 1, 2, \dots, N$

4. 拟合残差 r_{mi} 学习一个回归树，得到 $T(x; \theta_m)$, θ_m 为第 m 颗树的参数。

5. 更新 $f_m(x) = f_{m-1}(x) + T(x; \theta_m)$

5. end

输出：得到提升树 $f_M(x) = \sum_{m=1}^M T(x; \theta_m)$

➤ 举例：

接上面的例子：

$$\text{有：} T_1(x) = \begin{cases} 6.24, & x < 6.5 \\ 8.91, & x \geq 6.5 \end{cases}$$

$$f_1(x) = T_1(x)$$

用 $f_1(x)$ 拟合训练数据的残差，表中 $r_{2i} = y_i - f_{m-1}(x_i), i = 1, 2, \dots, 10$ ：

x	1	2	3	4	5	6	7	8	9	10
r_{2i}	-0.68	-0.54	-0.33	0.16	0.56	0.81	-0.01	-0.21	0.09	0.14

用 $f_1(x)$ 拟合训练数据的平方损失误差：

$$L(y, f(x_i)) = \sum_{i=1}^{10} (y_i - f(x_i))^2 = 1.93$$

➤ 举例：

第二步求 $T_2(x)$, 方法与求 $T_1(x)$ 一样, 只是拟合的数据是刚算出的残差, 可以得到：

$$T_2(x) = \begin{cases} -0.52, & x < 3.5 \\ 0.22, & x \geq 3.5 \end{cases}$$
$$f_2(x) = f_1(x) + T_2(x) = \begin{cases} 5.72, & x < 3.5 \\ 6.46, & 3.5 \leq x < 6.5 \\ 9.13, & x \geq 6.5 \end{cases}$$

用 $f_2(x)$ 拟合训练数据的平方损失误差是：

$$L(y, f_2(x)) = \sum_{i=1}^{10} (y_i - f_2(x_i))^2 = 0.79$$

继续求得 $T_3(x), T_4(x), T_5(x), T_6(x)$ 以及 $L(y, f_3(x)), L(y, f_4(x)), L(y, f_5(x))$ ：

$$T_3(x) = \begin{cases} 0.15, & x < 6.5 \\ -0.22, & x \geq 6.5 \end{cases} \longrightarrow L(y, f_3(x)) = 0.47$$
$$T_4(x) = \begin{cases} -0.16, & x < 4.5 \\ 0.11, & x \geq 4.5 \end{cases} \longrightarrow L(y, f_4(x)) = 0.30$$
$$T_5(x) = \begin{cases} 0.07, & x < 6.5 \\ -0.11, & x \geq 6.5 \end{cases} \longrightarrow L(y, f_5(x)) = 0.23$$
$$T_6(x) = \begin{cases} -0.15, & x < 2.5 \\ 0.04, & x \geq 2.5 \end{cases}$$

可以求得 $f_6(x) = f_5(x) + T_6(x) = T_1(x) + T_2(x) \dots + T_6(x)$

$$= \begin{cases} 5.63, & x < 2.5 \\ 5.82, & 2.5 \leq x < 3.5 \\ 6.56, & 3.5 \leq x < 4.5 \\ 6.83, & 4.5 \leq x < 6.5 \\ 8.96, & x \geq 6.5 \end{cases}$$

用 $f_6(x)$ 拟合训练数据的平方损失误差：

$$L(y, f_6(x)) = \sum_{i=1}^{10} (y_i - f_6(x_i))^2 = 0.17$$

假设此时已满足误差要求, 那么 $f(x) = f_6(x)$ 为所求提升树 (Decision Tree)。

XGBoost

XGBoost 目标函数 (每部分的意义和作用), 不需掌握目标函数构造过程以及切分节点等。

1. 数据: 维度为 p 的向量 $x \in \mathbb{R}^p$, 输出为 y , 定义数据集 D :

$$D = \{x_i, y_i\}_{i=1}^N$$

2. 模型: 基于数据集模型 $f(x)$ 需要做出预测, θ 为需要从数据中学习的参数:

$$\hat{y}_i = f(x_i; \theta)$$

3. 目标函数:

$$Obj(\theta) = \underbrace{L(\theta)}_{\text{损失函数}} + \underbrace{\Omega(\theta)}_{\text{正则化项}}$$

4. 学习/优化: 找到可以最小化目标函数的参数 θ

$$\theta^* = \arg \min_{\theta} Obj(\theta)$$

- 通过平衡损失函数与正则化项, 可以较好的拟合样本分布的同时增加模型的泛化性能
- 通过加入正则化项控制模型的复杂度, 防止过拟合, 从而提高模型的泛化性能:

神经网络

训练神经网络的基本步骤

(不需要掌握决策边界问题)

1. 数据: 维度为 p 的向量 $x \in \mathbb{R}^p$, 输出为 $y \in \{0, 1\}$, 定义数据集 D :

$$D = \{x_i, y_i\}_{i=1}^N$$

2. 模型: 基于数据集模型 $f(x)$ 需要做出预测, θ 为需要从数据中学习的参数:

$$\star \hat{y} = f_{\theta}(x_i)$$

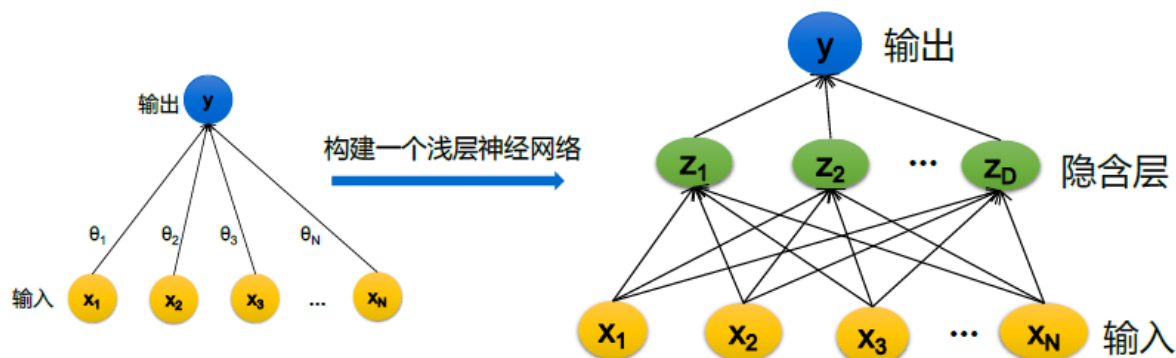
3. 目标函数:

$$J(\theta) = l(\hat{y}_i, y_i)$$

4. 优化: 通过梯度下降法求解参数

$$\star \theta^{(t+1)} = \theta^{(t)} - \eta_t \nabla \ell(f_{\theta}(x_i), y_i)$$

➤ 神经网络:



神经网络中的超参数及其作用

(网络宽度、深度、激活函数的选择对网络的影响，目标函数和初始化不需要掌握)

- **神经网络的宽度**

- 如果 $D \ll N$ ，隐含层作用类似于特征提取和降维
- 如果 $D \gg N$ ，隐含层作用类似于特征工程
- 足够的宽度可以保证每一层都学到丰富的特征，比如不同方向，不同频率的纹理特征。
- 宽度太窄，特征提取不充分，学习不到足够信息，模型性能受限。
- 太宽的网络会提取过多重复特征，加大模型计算负担。

- **神经网络的深度**

- 在多层神经网络中，第一个隐含层学习到的特征是最简单的，之后每个隐含层使用前一层得到的特征进行学习，所学到的特征变得越来越复杂
- Low level: 关注的是局部特征。
- Mid level: 学习中间特征。
- Top level: 更抽象的特征。

- **神经网络的激活函数**

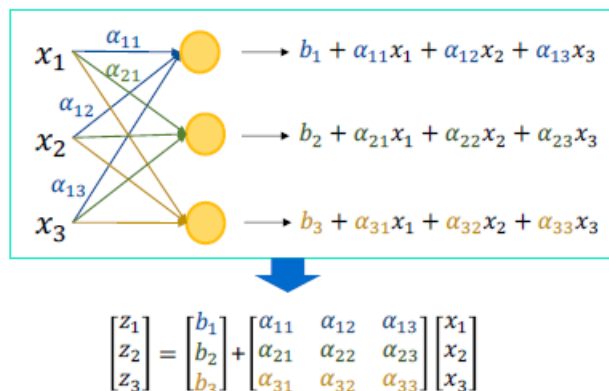
- 加入激活函数可以给模型**加入非线性因素，提升模型表达能力**
- Sigmoid 函数, 也就是logistic函数，对于任意输入，它的输出范围都是 (0,1)
- Tanh函数或双曲正切激活函数。类似于sigmoid 函数将输入转化到输出 (-1, 1) 范围内，缺点：梯度消失
- **修正线性单元 ReLU 函数** $Ax = \max(0, x)$
 - ReLU函数通过分段使得函数本身仍然是非线性的，不会减弱神经网络的表现能力
 - 2. 小于0时梯度为0使得部分神经元不会被激活，使得神经网络更加稀疏和高效
 - ReLU运算简单，比sigmoid函数更节约计算开销
 - ReLU函数同样也存在缺点，主要是由小于0的分段梯度同样为0造成，这意味着输出结果为负的神经元将停止对误差的响应，可能会导致许多神经元的“死亡”，通过使用ReLU函数的变体 PReLU (Parametric Rectified Linear Unit) 来解决。

神经网络向量化表示

两层神经网络（输入、隐藏层、输出层）的向量化表示（以Sigmoid为激活函数）

以3个输入的两层（一个隐藏层）的神经网络为例：

对于第一层:



写为矩阵形式:

$$z^{[1]} = \sigma(b^{[1]} + \alpha^{[1]} \cdot x)$$

对于第二层:

$$z^{[2]} = \sigma(\underset{\substack{\uparrow \\ \text{constant}}}{b^{[2]}} + \underset{\substack{\uparrow \\ 1 \times 3}}{\alpha^{[2]}} \cdot z^{[1]})$$

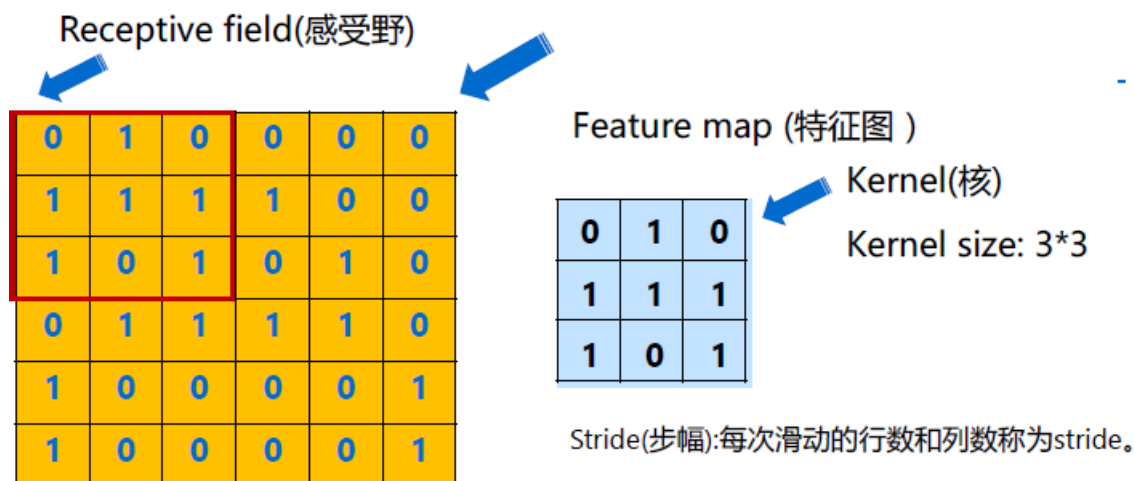
卷积神经网络

卷积神经网络的基本结构

卷积神经网络 (Convolutional Neural Network) 是含有卷积层 (Convolutional Layer) 的神经网络，具有深度结构的前馈神经网络是深度学习的代表算法之一，常用来处理图像数据。

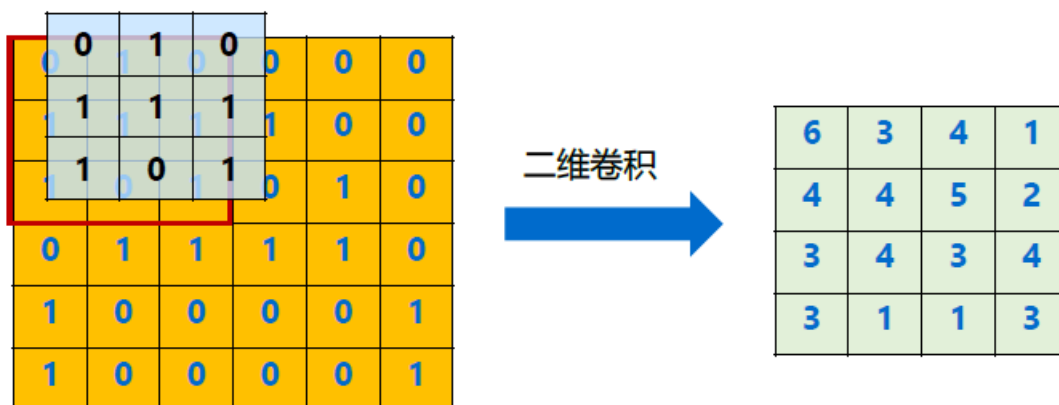
卷积神经网络的工作原理

感受野、特征图、Kernel、Padding、通道、池化、Flatten



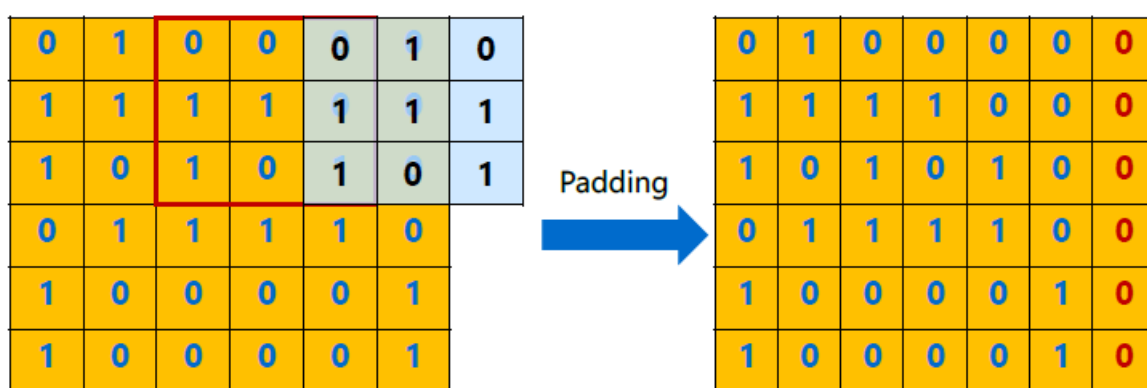
二维卷积运算：将输入域卷积核做互相关运算，从最左上方开始，按从左往右、从上往下的顺序，二维核窗口一次在输入数组上滑动，当窗口滑动到某一位置时，窗口中的输入数组与核数组按元素乘积并求和，得到输出数组中相应位置的元素。

➤ **Stride: 1 , Kernel size: 3*3**



假设输入的图像的尺寸是 $n_h \times n_w$, 卷积核的大小是 $k_h \times k_w$, 输出的大小为:
 $(n_h - k_h + 1) \times (n_w - k_w + 1)$

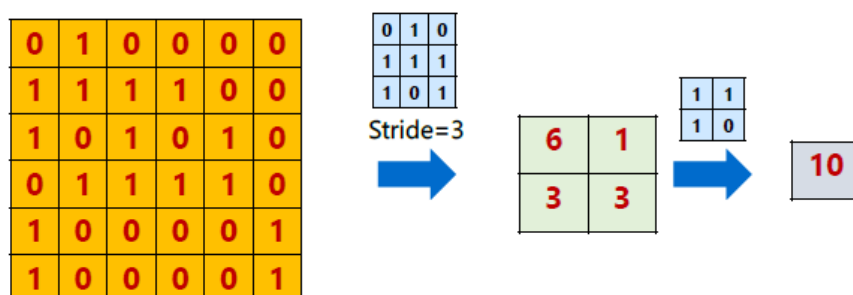
➤ **Stride: 2? , Kernel size: 3*3**



Padding (填充) : 在输入高和宽的两侧填充元素 ($p_w \times p_h$) (通常是0元素)。为了方便推测每个层的输出形状。通常会设置 $p_h = k_h - 1$ 和 $p_w = k_w - 1$ 来使输入和输出具有相同的大小。

局部连接：感受野

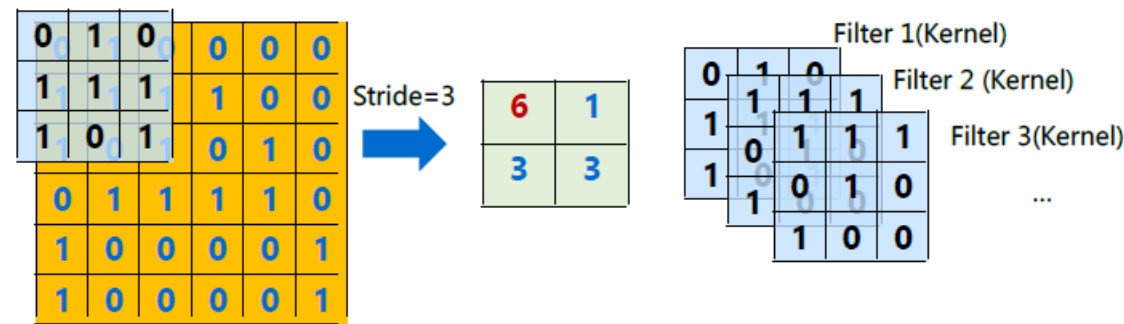
让每个神经元只与输入数据的一个局部区域连接 (感受野) , 会大大减少网络的参数, 同时可以提取到局部特征。



通过更深的卷积神经网络使特征图中单个元素的感受野变的更加广阔, 从而捕捉输入上更大尺寸的特征。

权值共享：卷积核

在卷积层中使用参数共享是用来控制参数的数量。每个滤波器与上一层局部连接，同时每个滤波器的所有局部连接都使用同样的参数，减少网络的参数。

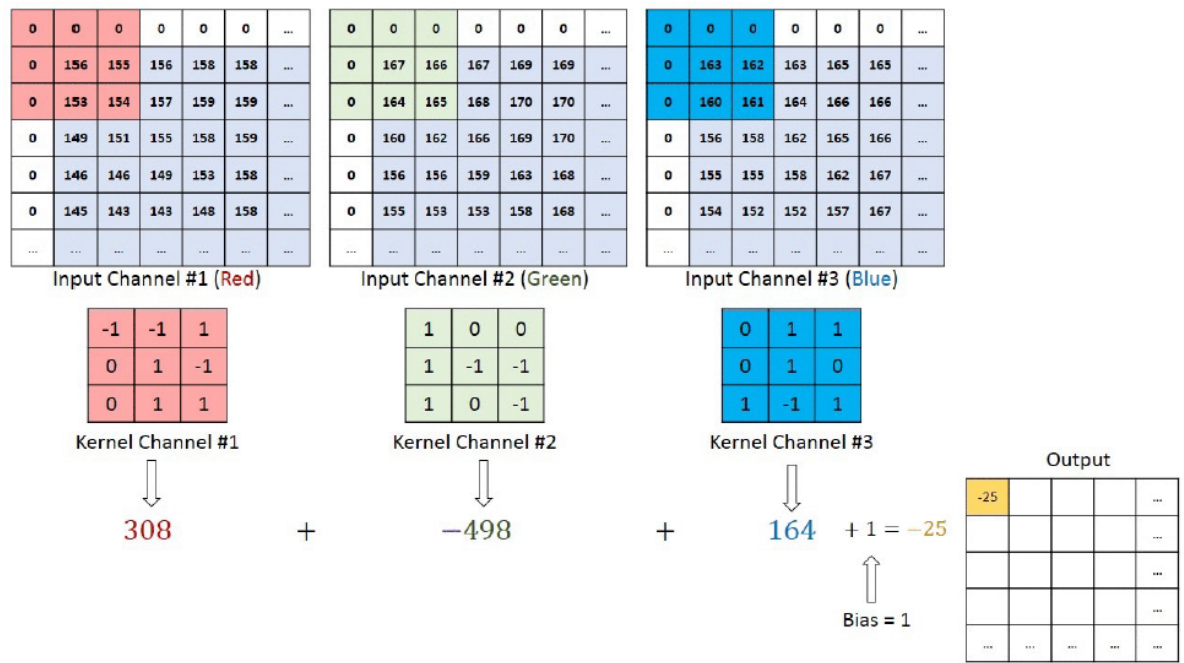


测试：

假设输入的图像的尺寸是 $n_h \times n_w$ ，卷积核的大小是 $k_h \times k_w$ ，输出的大小为，填充（Padding）的行为 p_h ，宽为 p_w ，输出的feature map的尺寸是多少：

$$(n_h - k_h + p_h + 1) \times (n_w - k_w + p_w + 1)$$

通道（Channel） --- 多通道卷积：每个通道跟一个卷积核做卷积运算，然后将结果相加得到一个特征图的输出。



池化

最大池化层

平均池化层

0	1	2
3	4	5
6	7	8

2*2池化

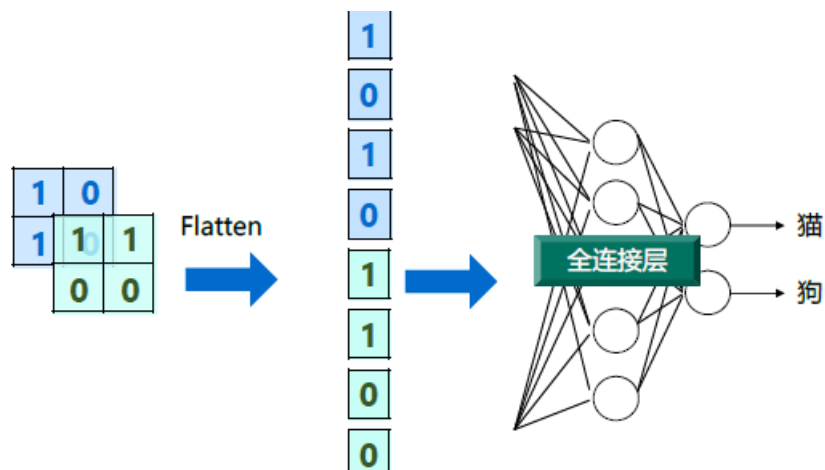
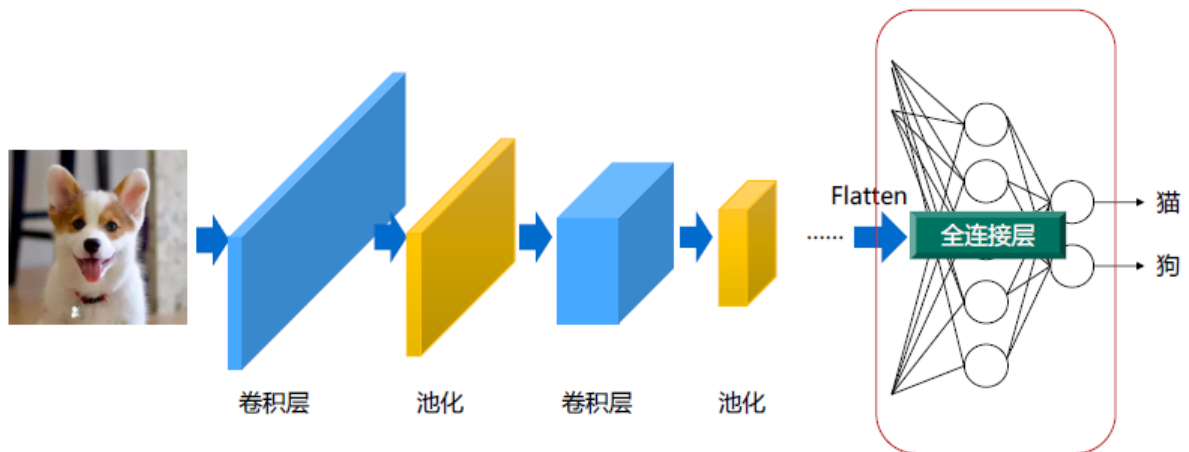
4	5
7	8

2	3
5	6

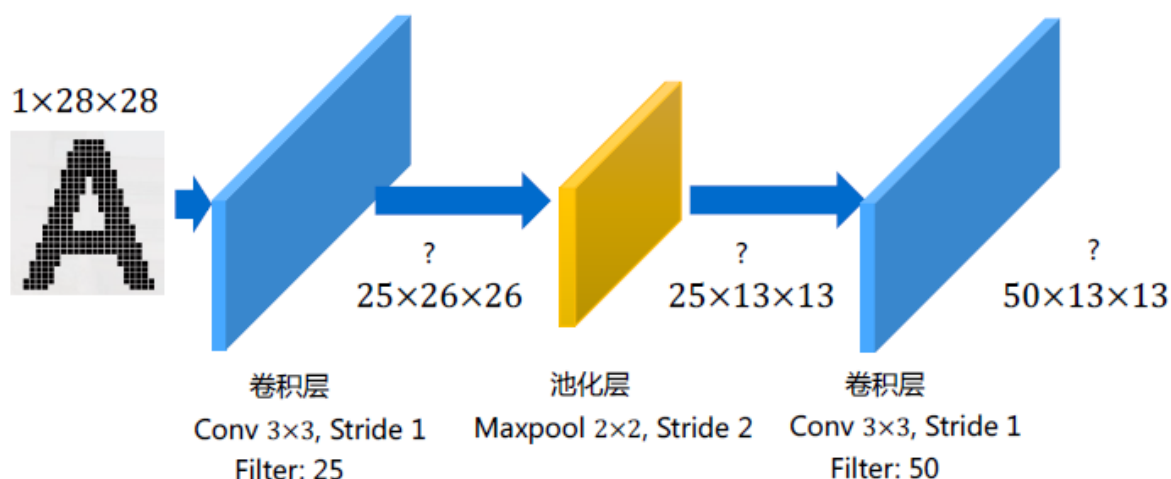
$$\begin{aligned}\max(0,1,3,4) &= 4 \\ \max(1,2,4,5) &= 5 \\ \max(3,4,6,7) &= 7 \\ \max(4,5,7,8) &= 8\end{aligned}$$

$$\begin{aligned}\text{mean}(0,1,3,4) &= 2 \\ \text{mean}(1,2,4,5) &= 3 \\ \text{mean}(3,4,6,7) &= 5 \\ \text{mean}(4,5,7,8) &= 6\end{aligned}$$

Flatten



➤ 测试：数据维度和参数：每一层输出的数据的通道数×长×款？



(最后是 $50 \times 11 \times 11$)

卷积神经网络输入输出维度变化

LeNet 和 CNN 的三个特性及其缺陷

LeNet 的特性

1. LeNet是早期成功的神经网络，包含了现代CNN网络的基本组件
2. 卷积层保留输入形状，使图像的像素在高和宽两个方向上的相关性均可能被有效识别
3. 卷积层通过滑动窗口将同一卷积核与不同位置的输入重复计算，从而避免参数尺寸过大

LeNet 缺陷

1. LeNet可以在小数据集上取得较好的成绩，但是在更大的数据集上表现并不尽如人意
2. 卷积层通过滑动窗口将同一卷积核与不同位置的输入重复计算，从而避免参数尺寸过大
3. 神经网络的加速硬件没有大量普及
4. 非凸优化算法和参数初始化等理论研究还不够深入，复杂神经网络训练困难

CNN 的三个特性

1. 局部感知
2. 下采样
3. 权值共享

AlexNet 的优点和特点

AlexNet使用了8层卷积神经网络，是一种端到端（End-to-End）的学习方法，通过模型学习特征。

优点

1. 更大的 Kernel size
2. 更大的 Stride
3. Avgpool $\rightarrow$ Maxpooling / Overlap pooling
4. 更大的 Pooling 窗口
5. 更多的输出 Channel

其他特点

1. AlexNet将Sigmoid激活函数改成了更加简单的ReLU激活函数。
2. AlexNet通过Dropout(丢弃法) 来控制全连接层的模型复杂度。
3. AlexNet引入了大量的图像增广的方法, 如翻转、裁剪和颜色变化, 进一步扩大数据集防止过拟合。
4. 没有进行图像预处理, End-to-End的模型。

总结

1. AlexNet不仅比LeNet的神经网络层数更多更深, 使用了更多的卷积层和更大的参数空间来拟合大规模数据集ImageNet, 是浅层神经网络和深度神经网络的分界线。
2. 可以学习更复杂的图像高维特征。

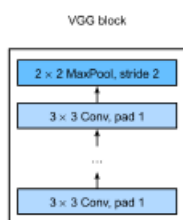
VGG架构, VGG块原理

VGG有两种结构, 分别是VGG16和VGG19, 两者并没有本质上的区别, 只是网络深度不一样。

原理

在VGG中使用3个3x3卷积核代替 7x7 (ZFNet) 卷积核, 使用2个3x3卷积核来代替5x5 (LeNet) 卷积核, 在保证具有相同感知野的条件下, 提升了网络的深度, 减小了网络参数, 在一定程度上提升了神经网络的效果。

VGG块 :



组成规律: 连续使用数个相同的填充为1、窗口形状为3x3的卷积层后接上一个步幅为2、窗口形状为2x2的最大池化层。卷积层保持输入的高和宽不变, 而池化层则对其减半。

VGG 架构

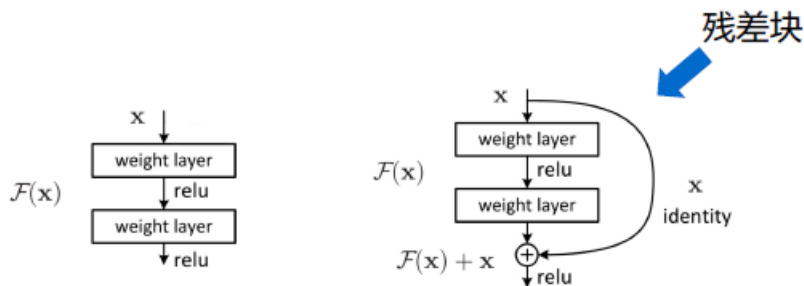
多个VGG块堆叠后接全连接层

总结

1. VGG结构简洁, 整个网络使用了同样大小的卷积核 (3x3) 和最大池化尺寸 (2x2) 。
2. 证明了小滤波器 (3x3) 卷积层组合比一个大滤波器 (5x5 或 7x7) 卷积层效果好。
3. 验证了不断加深网络结构可以提升性能。

ResNet 中残差连接解决的问题

残差链接：



- 设输入为 x ，如果期望的潜在映射为 $H(x)$ ，可以拟合的函数为 $F(x)$ 。与其让 $F(x)$ 直接学习潜在的映射，不如去学习残差 $H(x)-x$ ，即 $F(x):=H(x)-x$ ，这样原本的前向路径上就变成了 $F(x)+x$ ，用 $F(x)+x$ 来拟合 $H(x)$ 。
- 残差映射在实际中更容易优化。
- 残差块中的输入可以通过跨层的数据线（短路链接，Shortcut）更快的向前传播。

循环网络 RNN

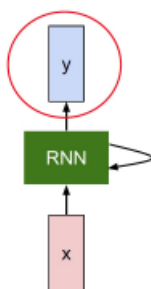
为什么需要循环神经网络

前馈网络的一些不足

1. 连接存在层与层之间，每层的节点之间是无连接的。（无循环）
2. 输入和输出的维数都是固定的，不能任意改变。无法处理变长的序列数据。
3. 假设每次输入都是独立的，也就是说每次网络的输出只依赖于当前的输入。
4. 但实际应用中，某些任务需要能够更好的处理序列信息，即前面的输入和后面的输入是有关系的

经典 (VanillaRNN) 结构

任务：预测在某一时间节点的输出向量

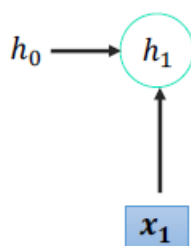


可以通过一个递归函数来对一个时序信息 x 进行建模：

$$h_t = g_{\theta}(h_{t-1}, x_t)$$

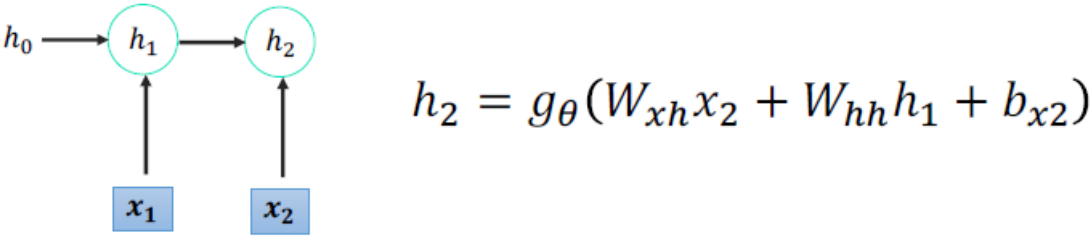
新的隐状态 非线性函数 旧的状态 某一时刻的输入

RNN引入了隐状态 h (hidden state)， h 可对序列数据提取特征，接着再转换为输出。首先从一个时间节点出发，先计算 h_1 ：

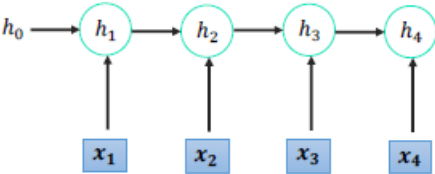


$$h_1 = g_{\theta}(W_{xh}x_1 + W_{hh}h_0 + b_{x1})$$

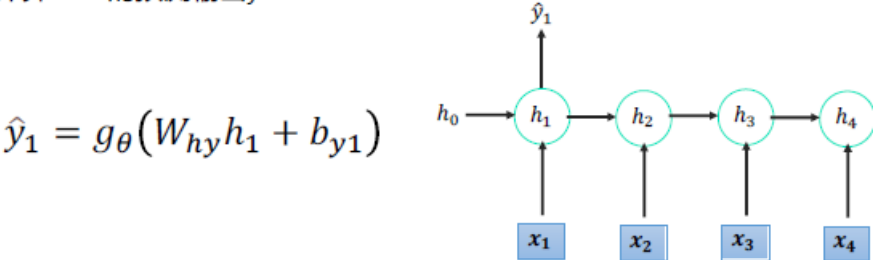
在第一步的基础上，上一步的隐状态 h_1 可以随时间传播到下一步用来计算 h_2 ，在RNN中每个步骤使用的参数 W_θ 都相同且共享，其计算结果如下：



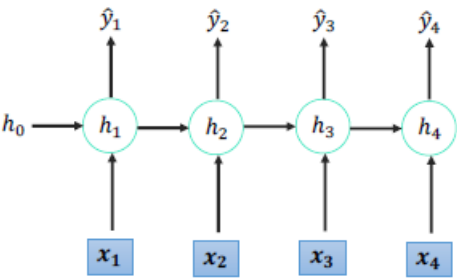
计算 h_3, h_4 ，网络结构如下图所示：



基于此，计算RNN的预测输出 y_1 ：



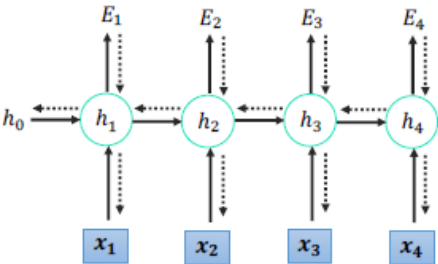
使用和 y_1 相同的参数，得到 y_1, y_2, y_3, y_4 的输出结构：



分析随时间反向传播 BPTT 的推导

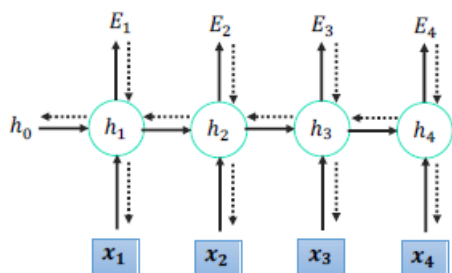
为什么会造成梯度消失和梯度爆炸

BPTT (back-propagation through time) 算法是常用的训练RNN的方法，其本质还是BP算法，只不过RNN处理时间序列数据，所以要基于时间反向传播，故叫随时间反向传播。



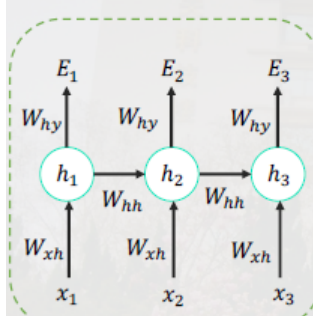
其中， E_t 是在时间步 t 上的损失函数，即RNN的输入与输出之间的误差，是 y_t 的函数，可以用 $E_t = \mathcal{L}(y_t)$ 表示。

整个模型在所有时间点的损失函数 $E = \sum_t E_t$ ，因此，可得 E 对于网络权重 W_θ 的梯度为



$$\begin{aligned}\frac{\partial E}{\partial W_\theta} &= \sum_{t=1}^T \frac{\partial E_t}{\partial W_\theta} = \sum_{t=1}^T \frac{\partial E_t}{\partial h_t} \frac{\partial h_t}{\partial W_\theta} \\ &= \sum_{t=1}^T \sum_{k=1}^t \frac{\partial E_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_\theta} \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_\theta}\end{aligned}$$

其中 $\frac{\partial h_k}{\partial W_\theta}$ 仅表示回传一步的偏导数，可以观察到，求解某一时刻的梯度，需要追溯这个时刻之前所有时刻的信息。



为了简化推导过程，假设只有三个时刻，那么在 $t=3$ 的时刻，对 W_θ 的偏导数分别为，在 $t=3$ 时刻的偏导数，需要追溯这个时刻之前所有时刻的信息：

$$\frac{\partial E_3}{\partial W_\theta} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial h_3} \frac{\partial h_3}{\partial W_\theta} + \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_\theta} + \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_\theta}$$

在 $t=2$ 时刻的偏导数

$$\frac{\partial E_2}{\partial W_\theta} = \frac{\partial E_2}{\partial \hat{y}_2} \frac{\partial \hat{y}_2}{\partial h_2} \frac{\partial h_2}{\partial W_\theta} + \frac{\partial E_2}{\partial \hat{y}_2} \frac{\partial \hat{y}_2}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_\theta}$$

在 $t=1$ 时刻的偏导数 $\frac{\partial E_1}{\partial W_\theta} = \frac{\partial E_1}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial h_1} \frac{\partial h_1}{\partial W_\theta}$

根据上面两个式子得出 E 在 t 时刻偏导数的通式：

$$\frac{\partial E_t}{\partial W} = \sum_t \frac{\partial E_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_t} \left(\prod_{j=k+1}^t \frac{\partial h^{(j)}}{\partial h^{(j-1)}} \right) \frac{\partial h^{(k)}}{\partial W}$$

整体的梯度更新公式还要将每一时刻加起来

$$\frac{\partial E}{\partial W_\theta} = \sum_{t=1}^T \frac{\partial E_t}{\partial W_\theta} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial E_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_\theta} \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_\theta}$$

$$\frac{\partial E}{\partial W_\theta} = \sum_{t=1}^T \frac{\partial E_t}{\partial W_\theta} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial E_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_\theta} \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_\theta}$$

进一步求解的关键在于求解 $\frac{\partial h_t}{\partial h_{t-1}}$ ，假设 h_t 和 h_{t-1} 都是 p 维列向量， $h_t[i]$ 为 h_t 第 i 维元素，可进一步写为

$$\begin{aligned}h_t &= [h_t[1], h_t[2], \dots, h_t[d]]^T \\ &= \begin{bmatrix} g_\theta(x_t, h_{t-1})[1] \\ g_\theta(x_t, h_{t-1})[2] \\ \vdots \\ g_\theta(x_t, h_{t-1})[p] \end{bmatrix} \\ &= \begin{bmatrix} g_\theta(W_{xh}[1]x_t + W_{hh}[1]h_{t-1}) \\ g_\theta(W_{xh}[2]x_t + W_{hh}[2]h_{t-1}) \\ \vdots \\ g_\theta(W_{xh}[p]x_t + W_{hh}[p]h_{t-1}) \end{bmatrix}\end{aligned}$$

$$\frac{\partial E}{\partial W_\theta} = \sum_{t=1}^T \frac{\partial E_t}{\partial W_\theta} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial E_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_\theta} \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_\theta}$$

进一步求导可得：

$$\begin{aligned} \frac{\partial h_t}{\partial h_{t-1}} &= \left[\frac{\partial g_\theta(x_t, h_{t-1})[1]}{\partial h_{t-1}}, \frac{\partial g_\theta(x_t, h_{t-1})[2]}{\partial h_{t-1}}, \dots, \frac{\partial g_\theta(x_t, h_{t-1})[p]}{\partial h_{t-1}} \right]^T \\ &= \begin{bmatrix} W_{hh}[1]g'_\theta(x_t, h_{t-1})[1] \\ W_{hh}[2]g'_\theta(x_t, h_{t-1})[2] \\ \vdots \\ W_{hh}[p]g'_\theta(x_t, h_{t-1})[p] \end{bmatrix} \\ &= \text{diag}[g'_\theta(x_t, h_{t-1})]W_{hh} \end{aligned}$$

其中 $\text{diag}[g'_\theta(x_t, h_{t-1})]$ 为对角阵，其对角元素为 $g'_\theta(x_t, h_{t-1})[i]$ 。

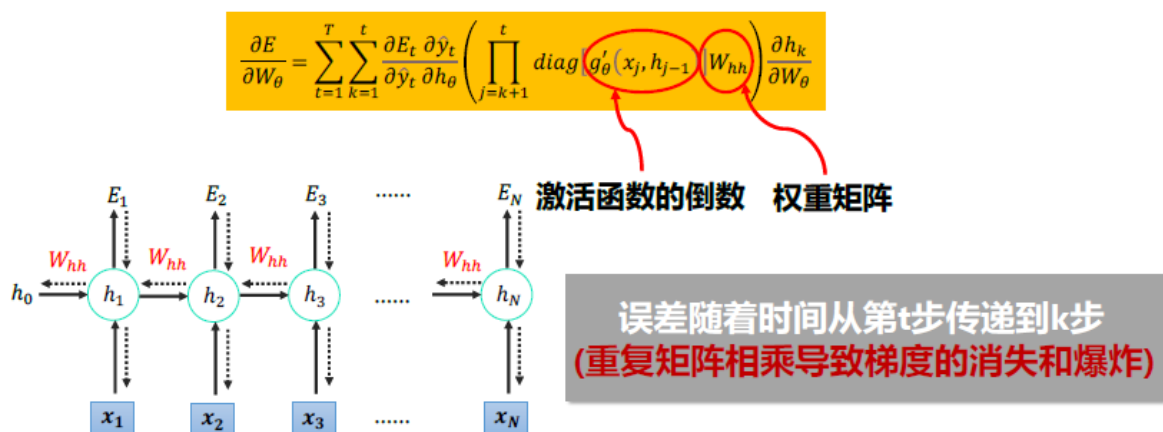
$$\frac{\partial E}{\partial W_\theta} = \sum_{t=1}^T \frac{\partial E_t}{\partial W_\theta} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial E_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_\theta} \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_\theta}$$

将上式带入到对于网络权重 W_θ 的梯度公式中，可得

$$\frac{\partial E}{\partial W_\theta} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial E_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_\theta} \left(\prod_{j=k+1}^t \text{diag}[g'_\theta(x_j, h_{j-1})]W_{hh} \right) \frac{\partial h_k}{\partial W_\theta}$$

RNN即是通过上式实现沿时间的反向传播。

梯度消失和梯度爆炸



假设在t=5的时刻：

$$\frac{\partial E_5}{\partial \theta} = \frac{\partial E_5}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial \theta} + \frac{\partial E_5}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial \theta} + \frac{\partial E_5}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial \theta} + \dots$$

$A = \begin{bmatrix} -1.70e-10 & 4.94e-10 & 2.29e-10 \\ -1.73e-10 & 5.56e-10 & 2.55e-10 \\ -1.81e-10 & 4.40e-10 & 2.08e-10 \end{bmatrix}$
 $\|A\| = 1.00e-09$

$B = \begin{bmatrix} -1.13e-08 & 2.61e-09 & 1.50e-08 \\ -1.11e-08 & 5.70e-09 & 1.51e-08 \\ -1.33e-08 & 9.11e-09 & 1.83e-08 \end{bmatrix}$
 $\|B\| = 1.53e-07$

$C = \begin{bmatrix} -1.70e-06 & 8.70e-06 & 9.40e-06 \\ -2.51e-07 & 7.30e-06 & 8.98e-06 \\ 7.32e-07 & 7.85e-06 & 1.05e-05 \end{bmatrix}$
 $\|C\| = 2.18e-05$

➤ $\frac{\partial E_5}{\partial \theta}$ is dominated by short-term dependencies (e.g., C), but

➤ Gradient **vanishes** in long-term dependencies i.e. $\frac{\partial E_5}{\partial \theta}$ is updated much less due to A as compared to updated by C

长时间序列的梯度项快速的呈指数型收敛到0，因此模型感受不到时间点间隔较长的信息。

假设在t=5的时刻：

$$\frac{\partial E_5}{\partial \theta} = \frac{\partial E_5}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial \theta} + \frac{\partial E_5}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial \theta} + \frac{\partial E_5}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial \theta} + \dots$$

$A = \begin{bmatrix} -1.70e+10 & 4.94e+10 & 2.29e-10 \\ -1.73e+10 & 5.56e+10 & 2.55e-10 \\ -1.81e+10 & 4.40e+10 & 2.08e-10 \end{bmatrix}$
 $\|A\| = 1.00e+109$

$B = \begin{bmatrix} -1.13e+08 & 2.61e+09 & 1.50e+08 \\ -1.11e+08 & 5.70e+09 & 1.51e+08 \\ -1.33e+08 & 9.11e+09 & 1.83e+08 \end{bmatrix}$
 $\|B\| = 1.53e+107$

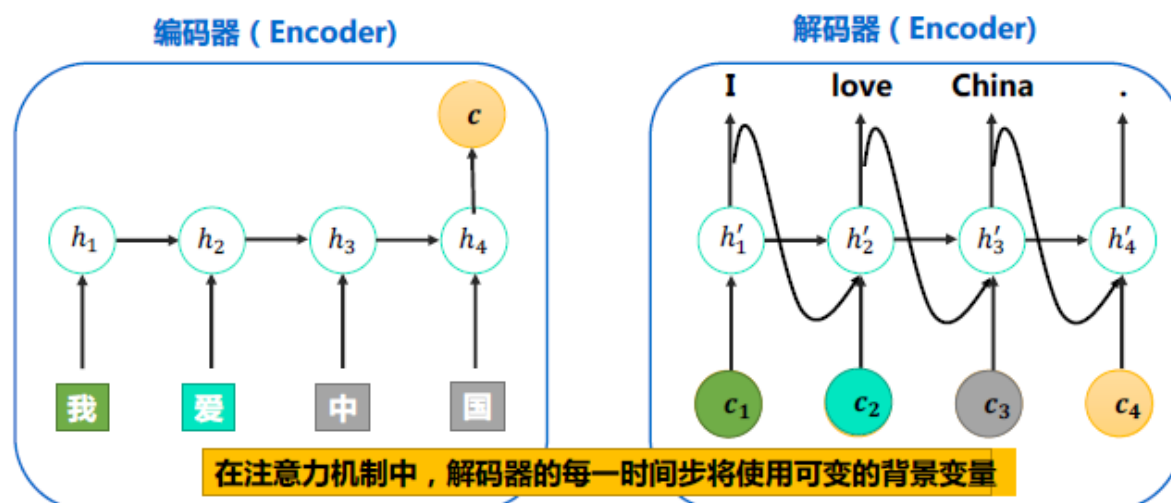
$C = \begin{bmatrix} -1.70e+06 & 8.70e+06 & 9.40e+06 \\ -2.51e+07 & 7.30e+06 & 8.98e+06 \\ 7.32e+07 & 7.85e+06 & 1.05e+05 \end{bmatrix}$
 $\|C\| = 2.18e+105$

长时间序列的梯度项快速的呈指数型增长，出现梯度爆炸。

Seq2seq 结构以及基础 Attention 机制

Seq2Seq

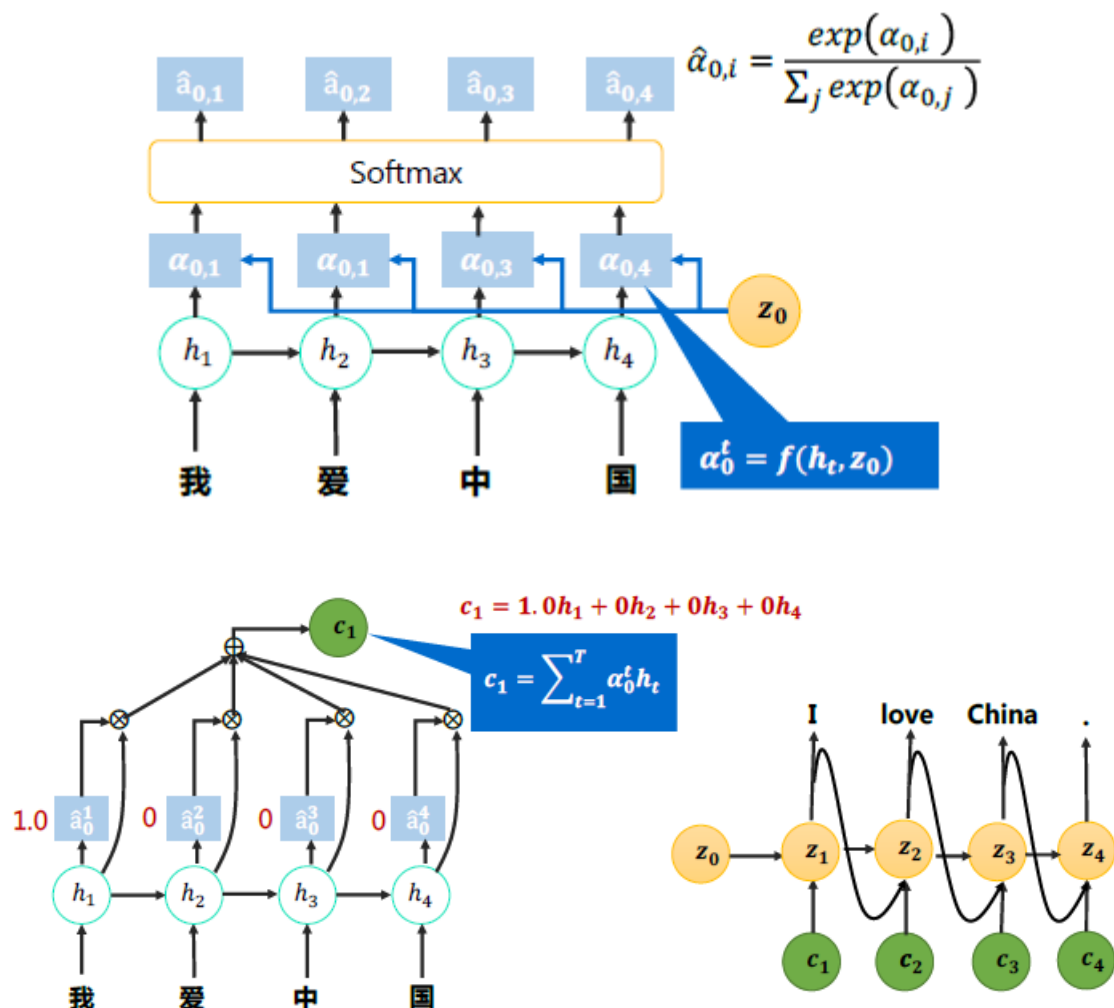
在自然语言处理的很多应用中，输入和输出都可以不是等长序列。在这种情况下，我们可以使用编码器-解码器模型 (Encoder--decoder) 或者叫 Seq2seq 模型。



Attention 机制

1. 解码器在生成输出序列中的每一个词可能只需要利用输入序列某一部分的信息。例如生成 I love 只需要看到前两个词。
2. 注意力机制通过对编码器所有时间步的隐藏状态做加权平均来得到背景变量。

3. 解码器在每一时间步调整这些权重，即注意力权重。
4. 在不同时间步关注输入序列中的不同部分并编码进相应时间步的背景变量。

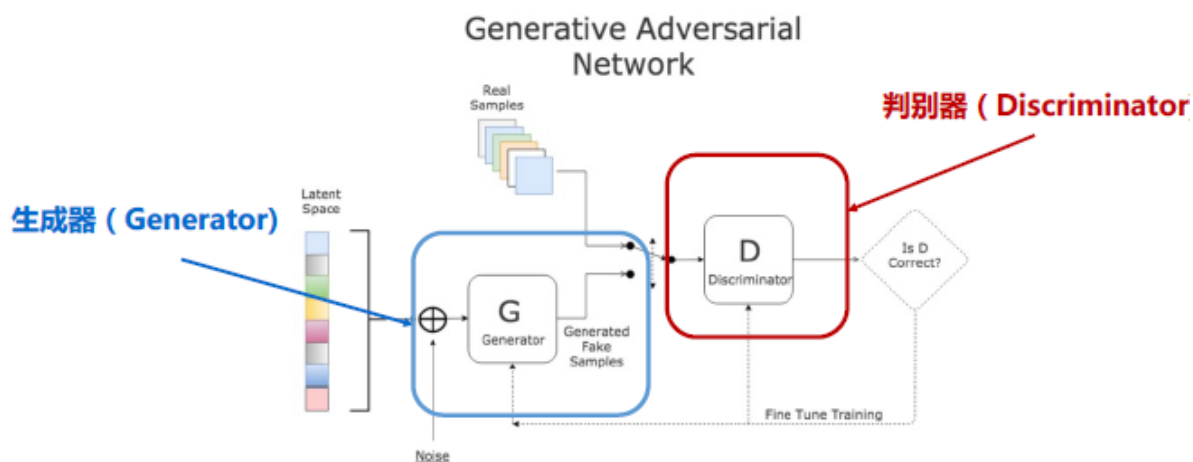


生成对抗网络

基本结构和各部分的作用

生成对抗网络GAN的基本结构和各部分的作用，生成器和判别器的输入和输出

GAN由两个主要网络构成：一个是 Generator Network (生成器)，一个是 Discriminator Network (判别器)。所有 GAN 网络的核心逻辑是生成器和判别器相互对抗、相互博弈。



GAN的目标函数和各部分的意义

GAN的目标函数如下：

$$\min_G \max_D V(D, G) = \min_G \max_D E_{x \sim P_{data}(x)} [\log D(x)] + E_{z \sim P_z(z)} [\log(1 - D(G(z)))]$$

从判别器角度来讲：如果 x 来自于 P_{data} ， $D(x; \theta_d)$ 的输出应该较大。反之，如果 x 来自于 P_g （ P_g 数据由 $z \sim P_z(z)$ 通过神经网络生成）， $D(x; \theta_d)$ 的输出应该较小（等价于 $1 - D(x; \theta_d)$ 较大），因此判别器的目标函数可写为：

$$\max_D E_{x \sim P_{data}(x)} [\log D(x)] + E_{z \sim P_z(z)} [\log(1 - D(G(z)))] \quad (1)$$

从生成器角度来讲：如果 x 来自于 P_z ，希望 $D(x; \theta_d)$ 的输出应该较大（等价于 $1 - D(x; \theta_d)$ 较小），因此生成器的目标函数可写为：

$$\min_G E_{z \sim P_z(z)} [\log(1 - D(G(z)))] \quad (2)$$

综合(1)(2)式，总目标函数可写为：

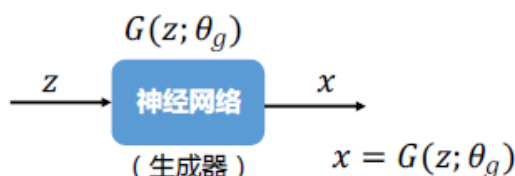
$$\min_G \max_D V(D, G) = \min_G \max_D E_{x \sim P_{data}(x)} [\log D(x)] + E_{z \sim P_z(z)} [\log(1 - D(G(z)))]$$

GAN的训练过程

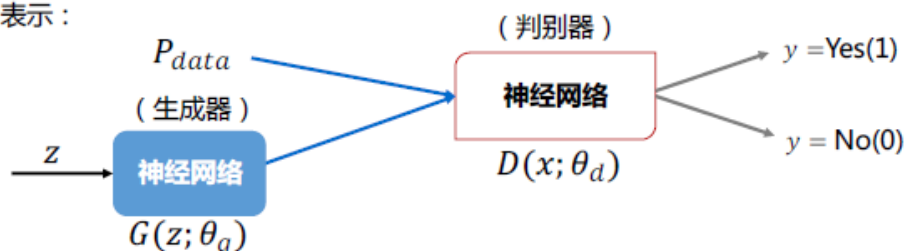
(不需掌握全局最优推导)

1) 从数据库中拿出真实样本 $\{x_i\}_{i=1}^N: P_{data}$

2) 生成器的生成数据服从分布 P_g ，在生成对抗网络中，不直接对 P_g 进行建模，而是通过一个神经网络去逼近这个分布，假定 z 表示噪声数据，服从分布 $z \sim P_z(z)$ （假设为随机噪声），将随机噪声输入生成器 G ，可用下图表示：



3) 判别器 $D(x; \theta_d)$ ， x 可以是真实数据也可以是生成器生成的数据， $D(x; \theta_d)$ 表示 x 是真实数据的概率，可用下图表示：



训练过程

1. 第一步希望判别器最大，那么此时固定 G 的参数，只训练 D ，将判别器的目标函数变换成积分形式：

$$\max_D V(D, G) = \int [P_{data}(x) \log D(x) + P_g(x) \log(1 - D(x))] dx$$

2. 要找到一个 D 使得 $V(D, G)$ 最大, 对其求偏导

$$\begin{aligned} \frac{\partial V(D, G)}{\partial D} &= \int \frac{\partial}{\partial D} [P_{data}(x) \log D(x) + P_g(x) \log(1 - D(x))] dx \\ &= 0 \\ \int [P_{data}(x) \frac{1}{D(x)} + P_g(x) \frac{-1}{1 - D(x)}] dx &= 0 \end{aligned}$$

3. 将上式进行变化可得

$$D^* = \frac{P_{data}(x)}{P_{data}(x) + P_g(x)}$$

4. 将推导出的公式带入目标函数中

$$\begin{aligned} \min_G \max_D V(D, G) &= \min_G V(D^*, G) \\ &= \min_G E_{x \sim P_{data}} [\log D^*(x)] + E_{x \sim P_g} [\log(1 - D^*(x))] \\ &= \min_G E_{x \sim P_{data}} [\log \frac{P_{data}(x)}{P_{data}(x) + P_g(x)}] + E_{x \sim P_g} [\log \frac{P_g(x)}{P_{data}(x) + P_g(x)}] \end{aligned}$$

5. 做一些简单的变换

$$\begin{aligned} &= \min_G E_{x \sim P_{data}} [\log \frac{\frac{1}{2} P_{data}(x)}{\frac{P_{data}(x) + P_g(x)}{2}}] + E_{x \sim P_g} [\log \frac{\frac{1}{2} P_g(x)}{\frac{P_{data}(x) + P_g(x)}{2}}] \\ &= \min_G -2 \log 2 + E_{x \sim P_{data}} [\log \frac{P_{data}(x)}{P_{data}(x) + P_g(x)}] + E_{x \sim P_g} [\log \frac{P_g(x)}{P_{data}(x) + P_g(x)}] \\ \min_G \max_D V(D, G) &= \min_G V(D^*, G) \\ &= \min_G -2 \log 2 + E_{x \sim P_{data}} [\log \frac{P_{data}(x)}{P_{data}(x) + P_g(x)}] + E_{x \sim P_g} [\log \frac{P_g(x)}{P_{data}(x) + P_g(x)}] \\ &= \min_G -\log 4 + \text{KL}(P_{data} \parallel \frac{P_{data}(x) + P_g(x)}{2}) + \text{KL}(P_g \parallel \frac{P_{data}(x) + P_g(x)}{2}) \\ &\geq -\log 4 \end{aligned}$$

KL散度 (相对熵) : $KL(P \parallel Q) = \int P(x) \log \frac{P(x)}{Q(x)} dx$

当 $P_{data} = \frac{P_{data}(x) + P_g(x)}{2} = P_g$ 时成立 :

$$\text{最优 } P_g^* = P_{data}, \text{ 此时 } D^* = \frac{P_{data}(x)}{P_{data}(x) + P_g(x)} = \frac{1}{2}$$

训练过程可总结如下：

1) 固定生成器 G ，训练判别器 D ，获得 $\max_D V(D, G)$ 。

2) 固定判别器 D ，训练生成器 G ，通过梯度下降求解更新参数：

$$\theta_g = \theta_g - \eta \frac{\partial \mathcal{L}(G)}{\partial \theta_g} = \theta_g - \eta \nabla \mathcal{L}(\theta_g)$$

3) 重复循环上面两步直到模型收敛。

传统GAN的缺陷以及其改进方法

(WGAN, WGAN-GP) 思想

传统GAN存在的问题

1. 梯队消失，JS散度的问题

解决方法：

1. 不把判别器训练的太好。

2. 给生成数据和真实数据加噪声，强行让生成数据与真实数据在高维空间产生重叠，JS散度就可以发挥作用：

$$\min_G E_{x \sim P_{data+\epsilon}} [\log D^*(x)] + E_{x \sim P_{g+\epsilon}} [\log(1 - D^*(x))]$$

$P_{data+\epsilon}$ 表示加入噪声后的真实分布数据， $P_{g+\epsilon}$ 表示加入噪声后的生成分布数据。

2. 模式崩溃(mode collapse)

解决方法：

1) 从目标函数考虑：当GAN出现模式崩溃问题时，通常判别器在训练样本附近更新参数时，其梯度值非常大。可对判别器在训练样本附近施加梯度惩罚项。试图在训练样本附近构建线性函数，因为线性函数为凸函数具有全局最优解。(DRAGAN)

2) 从网络架构考虑：即使单个生成器会产生模式崩溃的问题，但是如果同时构造多个生成器，且让每个生成器产生不同的模式，则这样的多生成器结合起来也可以保证产生的样本具有多样性。(MAD-GAN)

WGAN总结，与传统GAN的区别：

1. 判别器最后一层去掉sigmoid。
2. 生成器与判别器的损失不再取log。
3. 训练判别器时，每次参数更新后的值在一个范围 $(-c, c)$
4. 推荐使用 RMSProp 或 SDG 优化算法。

WGAN-GP (GP: Gradient Penalty) 的提出就是为了解决 WGAN 遇到的问题，不再使用 WGAN 中 Weight clipping 的方式来粗暴地限制参数范围。

WGAN-GP：当判别器 D 服从1-Lipschitz约束时，等价于判别器 D 在任意地方的梯度都小于1，公式如下：

$$D \in 1 - \text{Lipschitz} \Leftrightarrow \|\nabla_x D(x)\| \leq 1$$

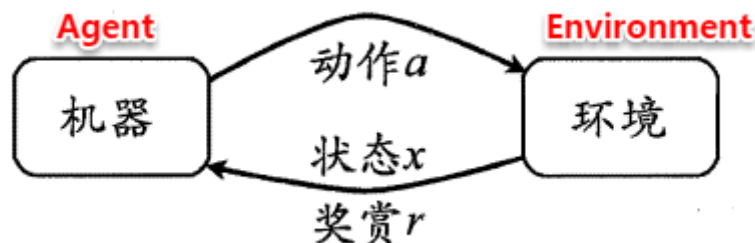
判别器的目标函数可改写为：

$$\max_D E_{x \sim P_{data}}[D(x)] - E_{x \sim P_g}[D(x)] - \lambda \int_x \max(0, \|\nabla_x D(x)\| - 1) dx$$

强化学习

强化学习中的基本概念

任务与奖赏、马尔科夫决策过程、强化学习的主要任务及与有监督学习的差异



马尔科夫决策过程描述强化学习任务

在一个环境 E 中，每个**状态**为机器对当前环境的感知；机器只能通过**动作**来影响环境，当机器执行一个动作后，会使得环境按某种**概率转移**到另一个状态；同时，环境会根据潜在的奖赏函数反馈给机器一个**奖赏**

- 状态 X
 - 对环境的感知，所有可能的状态称为状态空间.例如西瓜长势的描述
- 动作 A
 - 机器所采取的动作，所有能采取的动作构成动作空间。例如浇水、施肥等可选的动作
- 转移概率 P
 - 当执行某个动作后，当前状态会以某种概率转移到另一个状态。例如西瓜缺水，在浇水动作后，瓜苗长势发生变化
- 奖赏函数 R
 - 当在状态转移的同时，环境给反馈给机器一个奖赏。例如瓜苗健康+1,瓜苗凋零-10

强化学习的主要任务

通过在环境中不断地尝试，根据尝试获得的反馈信息调整策略，最终生成一个较好的策略 π ，机器根据这个策略便能知道在什么状态下执行什么动作

学习目的：找到长期累积奖赏**最大化**的策略。

与有监督学习的差异

1. 有时间相关性，不服从iid
2. 没有强烈的监督者，只有奖励信号
3. 奖励是延迟的，如游戏结束才知道动作好坏
4. 通过不断地试错探索才能获知选出最佳动作的能力
5. 强化学习是“延迟标记信息”的监督学习问题

□ 监督学习训练过程 不同：采样分布要求；标注要求；目标；

- 输入数据 x_i ，其服从iid数据分布
- 使用模型（参数为 w ）预测输出 y
- 观测目标输出 y_i 和损失 $l(w, x_i, y_i)$
- 更新 w 以降低损失 $w \leftarrow w - \eta \nabla l(w, x_i, y_i)$



□ 强化学习训练过程

分类器 策略 示例 状态 标记 动作

- 从状态 x ，采取由策略 $\pi(x)$ 确定的行动 a
- 环境基于转移模型 $P(x'|x, a)$ 选择下一状态 x'
- 观测状态 x' 和奖励 $r(x')$
- 更新策略



探索和利用、 ϵ 贪心算法过程

探索和利用

单步强化学习实质上是K-摇臂赌博机的原型，一般我们尝试动作的次数是有限的，那如何利用有限的次数进行有效地探索呢？这里有两种基本的想法：

1. 仅探索法：将所有的尝试机会平均分配给每个摇臂(即轮流按下每个摇臂)，即轮流执行，最后以每个摇臂各自的平均吐币概率作为其奖赏期望的近似估计
 - 很好地估计每个摇臂的奖赏，却会失去很多选择最优摇臂的机会
2. 仅利用法：按下目前最优的(即到目前为止平均奖赏最大的)摇臂，若有多个摇臂同为最优，则从中随机选取一个
 - 没有很好地估计摇臂期望奖赏，很可能经常选不到最优摇臂

想要累积奖赏最大，则必须在探索与利用之间达成较好的折中。

ϵ -贪心

- 以 ϵ 的概率进行探索：即以均匀概率随机选择一个动作
- 以 $1-\epsilon$ 的概率进行利用：即选择当前最优的动作
- 若奖赏不确定性大，例如概率分布较宽时，则需更多的探索，设置较大的 ϵ
- 若奖赏不确定性小，例如概率分布较集中时，则少量的尝试就可很好地近似真实奖赏，需要较小的 ϵ
- 通常 ϵ 取0.1或0.01

有模型学习中的值迭代与策略迭代的算法过程及差异

策略迭代 (以T步为例)

1. 先给定一个随机策略为初始策略
2. 策略评估
3. 改进策略
4. 循环以上评估/改进过程，直到策略收敛，不再改变为止

缺点：每次改进策略后都需重新进行策略评估，比较耗时

值迭代 (以T步为例)

1. 先给定一个随机策略为初始策略
2. 策略评估和改进策略（**隐含进行**）同时完成（选最优动作）
3. 循环以上评估/改进过程，直到值函数收敛
4. 根据最优值函数改变策略

策略评估

模型已知时，可对任意策略 π 估计出其带来的**期望累积奖赏**，进而评估策略优劣

- 状态值函数 $V(\cdot)$ ：从状态 x 出发，使用策略 π 获得的累积奖赏 $V^\pi(x)$
- 状态-动作值函数 $Q(\cdot)$ ：从状态 x 出发，执行动作 a 后，再使用策略 π 获得的累积奖赏 $Q^\pi(x, a)$

策略改进

最优策略：应能使得所有状态的累积奖赏之和最大，即不管当前处于什么状态，只要通过该策略执行动作，均可获得很好的结果。

对某个策略进行评估后，即计算其对应的累计奖赏并非最大，该策略非最优策略，应对其进行改进

改进后的策略可得到最优的值函数 $\forall x \in X : V^*(x) = V^{\pi^*}(x)$

免模型学习中的蒙特卡罗算法思想

在现实的强化学习任务中，环境的**转移函数与奖赏函数往往很难得知**，因此我们需要考虑在**不依赖于环境参数的条件下**建立强化学习模型，这便是免模型学习

综合策略（蒙特卡罗强化学习）：从起始状态出发，使用某种策略采样获得T步的采样轨迹，对其中的每一对状态-动作，记录其后的奖赏值之和，作为该状态-动作的一次累积奖赏：

$$\langle x_0, a_0, r_1, x_1, a_1, r_2, \dots, x_{T-1}, a_{T-1}, r_T, x_T \rangle$$

状态-动作值函数：由多条采样轨迹对每个状态-动作对的累计奖赏采样值进行平均

□ 对策略评估：使用策略 π 的采样轨迹评估策略 π ，就是对累积奖赏求期望

$$Q(x, a) = \frac{1}{N} \sum_{i=1}^N R_i \quad R_i \text{ 为第 } i \text{ 条轨迹从状态 } x \text{ 至结束的累积奖赏}$$

□ 按重要性采样原理：使用策略 π' 的采样轨迹评估策略 π ，仅需对累积奖赏加权

$$Q(x, a) = \frac{1}{N} \sum_{i=1}^N \frac{p_i^\pi}{p_i^{\pi'}} R_i \quad p_i^\pi, p_i^{\pi'} \text{ 分别为两个策略产生第 } i \text{ 条轨迹的概率}$$

□ 对给定的轨迹 $\langle x_0, a_0, r_1, x_1, a_1, r_2, \dots, x_{T-1}, a_{T-1}, r_T, x_T \rangle$ ，策略 π 产生该轨迹的概率为

$$p^\pi = \prod_{i=0}^{T-1} \pi(x_i, a_i) p_{x_i \rightarrow x_{i+1}}^{a_i}$$

□ 上式用到了转移概率，但实际我们只需要两个策略概率的比值

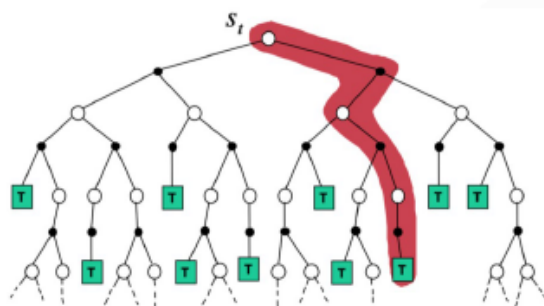
$$\frac{p^\pi}{p^{\pi'}} = \prod_{i=0}^{T-1} \frac{\pi(x_i, a_i)}{\pi'(x_i, a_i)}$$

蒙特卡罗算法

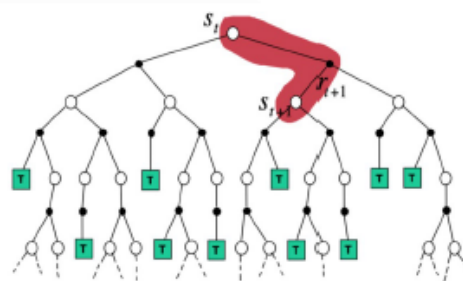
- 采样轨迹的方式克服了模型未知而难以进行策略估计的困难
- 必须在一个完整的采样轨迹后再更新
- 未使用MDP结构

□ 蒙特卡罗和时序差分：

$$Q_{t+1}^\pi(x, a) = Q_t^\pi(x, a) + \alpha(R_{x \rightarrow x'}^a + \gamma Q_t^\pi(x', a') - Q_t^\pi(x, a))$$



蒙特卡罗法一步更新整条路径上的状态



时序差分一步更新只影响局部状态

半监督学习

半监督学习的基本概念和假设

有监督的学习：学习器通过对大量有标记的训练例进行学习，从而建立模型用于预测未见示例的标记(label)。**很难获得大量的标记样本。**

无监督的学习：无训练样本，仅根据测试样本的在特征空间分布情况来进行标记，**准确性差。**

半监督的学习：有少量训练样本，学习机以从训练样本获得的知识为基础，结合测试样本的分布情况逐步修正已有知识，并判断测试样本的类别。

半监督学习特点

- 有标记数据少，无标记数据多
- 利用数据分布与类别标记相联系的假设
- 性能依赖于所用的半监督假设

数据分布信息与类别标记关系的假设：相似的样本有相似的输出

- **聚类假设：**同一聚类中的样本点很可能具有同样的类别标记。**关注样本空间的整体特征**，探测样本分布稠密和稀疏的区域，从而更好地约束决策边界穿过稀疏区域
- **流形假设：**处于一个很小的局部流形邻域内的示例具有相似的性质。利用大量的无标签样增加样本空间的密度，**从而更准确地获取样本的局部近邻关系**

生成式方法、半监督SVM对应的建模方法、优化过程及算法

生成式方法

假设

所有样本数据（无论是否标记）服从一个潜在的分布。该分布由先验知识作出假设。以高斯混合模型为例，假定样本的概率由多个高斯分布组合形成，从而一个子高斯分布就代表一个类簇（类别）。数据样本的概率密度如下

$$p(x) = \sum_{i=1}^N \alpha_i \cdot p(x | \mu_i, \Sigma_i) \quad \alpha_i \geq 0, \sum_{i=1}^k \alpha_i = 1$$

混合系数 均值向量 协方差矩阵

高斯分布子分量：
$$p(x | \mu_i, \Sigma_i) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma_i|^{\frac{1}{2}}} e^{-\frac{1}{2}(x-\mu_i)^T \Sigma_i^{-1} (x-\mu_i)}$$

分类

则可以根据有标签样本和无标签样本估计分布的参数，进而由后验概率判别类别

$$\begin{aligned}
 f(\mathbf{x}) &= \operatorname{argmax}_{j \in \mathcal{Y}} p(y = j | \mathbf{x}) \\
 &= \operatorname{argmax}_{j \in \mathcal{Y}} \sum_{i=1}^k p(y = j, \Theta = i | \mathbf{x}) \quad p(y = j | \Theta = i) \\
 &= \operatorname{argmax}_{j \in \mathcal{Y}} \sum_{i=1}^k p(y = j | \Theta = i, \mathbf{x}) \cdot p(\Theta = i | \mathbf{x})
 \end{aligned}$$

仅与样本 $\mathbf{x}$ 所属的高斯成分有关

其中属于第 i 个高斯成分的概率:

$$p(\Theta = i | \mathbf{x}) = \frac{\alpha_i p(\mathbf{x} | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)}{\sum_{i=1}^k \alpha_i p(\mathbf{x} | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)}$$

参数估计

根据所有样本极大似然法估计高斯混合模型的参数

$$D = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_l, y_l), \mathbf{x}_{l+1}, \dots, \mathbf{x}_m\}$$

$$LL(D_l \cup D_u) = \sum_{(\mathbf{x}_j, y_j) \in D_l} \ln \left(\sum_{i=1}^N \alpha_i \cdot p(\mathbf{x}_j | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) \cdot \underbrace{p(y_j | \Theta = i, \mathbf{x}_j)}_{\text{当且仅当 } i=j \text{ 时取 } 1} \right) + \sum_{\mathbf{x}_j \in D_u} \ln \left(\sum_{i=1}^N \alpha_i \cdot p(\mathbf{x}_j | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) \right)$$

有类标样本只在特定类簇中出现
无类标样本可能在所有类簇中出现

迭代优化

基于标记数据的有监督项和无标记数据的无监督项以EM算法求解

- 期望步(E步)：未标记样本属于各高斯混合成分的概率

$$\gamma_{ji} = \frac{\alpha_i \cdot p(\mathbf{x}_j | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)}{\sum_{i=1}^N \alpha_i \cdot p(\mathbf{x}_j | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)}$$

- 最大化期望(M步)：更新模型参数， l_{ii} 表示第 i 类的有标记样本数目

$$\boldsymbol{\mu}_i = \frac{1}{\sum_{\mathbf{x}_j \in D_u} \gamma_{ji} + l_i} \left(\sum_{\mathbf{x}_j \in D_u} \gamma_{ji} \mathbf{x}_j + \sum_{(\mathbf{x}_j, y_j) \in D_l \wedge y_j = i} \mathbf{x}_j \right) \quad \text{li指的是第 } i \text{ 类有标记样本数目}$$

$$\boldsymbol{\Sigma}_i = \frac{1}{\sum_{\mathbf{x}_j \in D_u} \gamma_{ji} + l_i} \left(\sum_{\mathbf{x}_j \in D_u} \gamma_{ji} (\mathbf{x}_j - \boldsymbol{\mu}_i)(\mathbf{x}_j - \boldsymbol{\mu}_i)^T + \sum_{(\mathbf{x}_j, y_j) \in D_l \wedge y_j = i} (\mathbf{x}_j - \boldsymbol{\mu}_i)(\mathbf{x}_j - \boldsymbol{\mu}_i)^T \right)$$

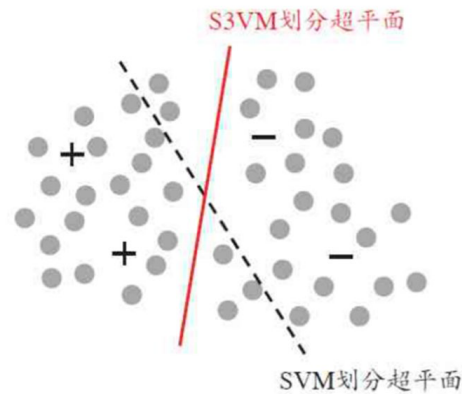
$$\alpha_i = \frac{1}{m} \left(\sum_{\mathbf{x}_j \in D_u} \gamma_{ji} + l_i \right)$$

半监督SVM

TSVM法利用局部搜索异类错分未标记样本迭代求近似解

考虑未标记样本

目标是找到最大间隔划分超平面、且穿过数据低密度区域将标记样本分开



TSVM (Transductive SVM)

尝试为未标记样本找到合适的标记指派，使得超平面划分后的间隔最大化

$$D = \{(x_1, y_1), \dots, (x_l, y_l), x_{l+1}, \dots, x_m\}$$

TSVM目标

为 D_u 中的样本给出预测标记

$$\hat{y} = (\hat{y}_{l+1}, \hat{y}_{l+2}, \dots, \hat{y}_m), \hat{y}_i \in \{-1, +1\}, \text{满足:}$$

$$\begin{aligned} \min_{w, b, \hat{y}, \xi} \quad & \frac{1}{2} \|w\|_2^2 + C_l \sum_{i=1}^l \xi_i + C_u \sum_{i=l+1}^m \xi_i \rightarrow \text{松弛变量 hinge损失} \\ \text{s.t.} \quad & y_i(w^T x_i + b) \geq 1 - \xi_i, \quad i = 1, 2, \dots, l, \\ & \hat{y}_i(w^T x_i + b) \geq 1 - \xi_i, \quad i = l+1, l+2, \dots, m, \\ & \xi_i \geq 0, \quad i = 1, 2, \dots, m, \end{aligned}$$

对应标记样本

对应未标记样本

TSVM求解

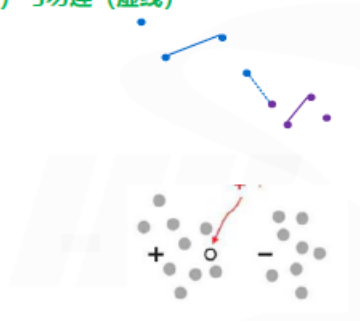
迭代地进行局部搜索。分两个环节：分配伪标记并获得初始SVM、交换错分的未标记样本更新SVM

约束K-均值聚类算法过程

拥有部分额外监督信息时：可利用监督信息改善聚类效果。

● 约束：必连（实线）与勿连（虚线）

● 少量有标签样本



输入：样本集 $D = \{x_1, x_2, \dots, x_m\}$;

必连约束集合 $\mathcal{M}$;

勿连约束集合 $\mathcal{C}$;

聚类簇数 k .

过程:

```
1: 从  $D$  中随机选取  $k$  个样本作为初始均值向量  $\{\mu_1, \mu_2, \dots, \mu_k\}$ ;  
2: repeat  
3:    $C_j = \emptyset$  ( $1 \leq j \leq k$ );  
4:   for  $i = 1, 2, \dots, m$  do  
5:     计算样本  $x_i$  与各均值向量  $\mu_j$  ( $1 \leq j \leq k$ ) 的距离:  $d_{ij} = \|x_i - \mu_j\|_2$ ;  
6:      $\mathcal{K} = \{1, 2, \dots, k\}$ ;  
7:     is_merged=false;  
8:     while  $\neg$  is_merged do  
9:       基于  $\mathcal{K}$  找出与样本  $x_i$  距离最近的簇:  $r = \arg \min_{j \in \mathcal{K}} d_{ij}$ ;  
10:      检测将  $x_i$  划入聚类簇  $C_r$  是否会违背  $\mathcal{M}$  与  $\mathcal{C}$  中的约束;  
11:      if  $\neg$  is_violated then  
12:         $C_r = C_r \cup \{x_i\}$ ;  
13:        is_merged=true  
14:      else  
15:         $\mathcal{K} = \mathcal{K} \setminus \{r\}$ ; 若不满足则寻找距离次小的类簇  
16:        if  $\mathcal{K} = \emptyset$  then  
17:          break并返回错误提示  
18:        end if  
19:      end if  
20:    end while  
21:  end for  
22:  for  $j = 1, 2, \dots, k$  do  
23:     $\mu_j = \frac{1}{|C_j|} \sum_{x \in C_j} x$ ;  
24:  end for  
25: until 均值向量均未更新  
输出: 簇划分  $\{C_1, C_2, \dots, C_k\}$ 
```

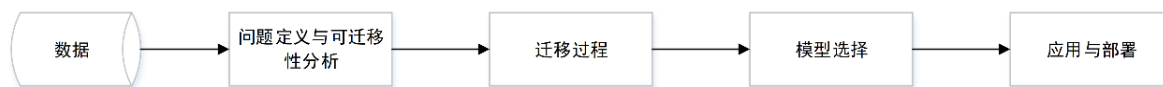
对样本进行划分时，需检测是否满足约束关系，其它步骤均相同

迁移学习

迁移学习问题的概念

概念

指利用数据、任务、或模型之间的相似性，将在旧领域学习过的模型，应用于新领域的一种学习过程。



迁移学习作用

- 减少对标记数据的依赖
- 降低对机器算力的需求
- 使模型适配个性化任务

域适应的概念

迁移学习一般可形式化为域适应问题

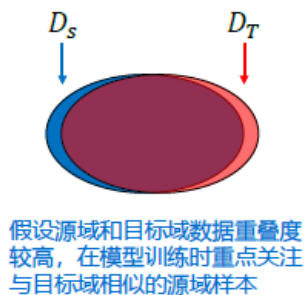
- 定义：以分类问题为例，域适应的定义如下：

给定源域数据集 $D_s = \{x_i, y_i\}_{i=1}^n$ （包括数据和标签），以及目标域数据集 $D_t = \{x_j\}_{j=n+1}^{n+m}$ （往往不含标签），利用 D_s 和 D_t 学习一个分类器 $f: x_t \rightarrow y_t$ ，该分类器输入目标域的数据，输出对应的分类结果。



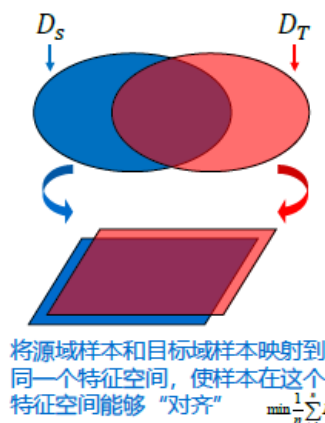
□ 现有域适应方法大致分为三类

- 基于实例的域适应



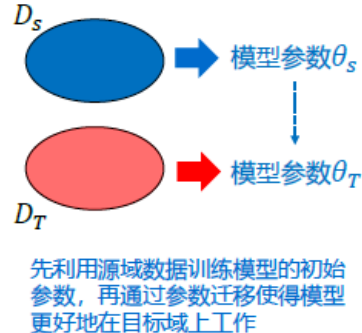
$$\min \frac{1}{n} \sum_{i=1}^n w_i L(\Phi(x_i^s), y_i^s, \theta)$$

- 基于特征的域适应 (目前最常用的方法)



$$\min \frac{1}{n} \sum_{i=1}^n L(\Phi(x_i^s), y_i^s, \theta)$$

- 基于模型参数的域适应



$$\min \frac{1}{n} \sum_{i=1}^n L(\Phi(x_i^s), y_i^s, \theta)$$

最大均值差异法的过程

(不需要掌握具体的推导)

最大均值差异：可度量特征空间中两个分布的距离

● 最大均值差异的一般形式：

$$\text{MMD}^2(A, B) = \left\| \sum_{i=1}^{n1} \phi(a_i) - \sum_{j=1}^{n2} \phi(b_j) \right\|^2$$

A, B : 两个样本集合 a_i : 集合A中的样本 b_j : 集合B中的样本
 ϕ : 特征映射函数 $\phi(b_j)$: 样本 b_j 的映射特征
 $\phi(a_i)$: 样本 a_i 的映射特征

直观解释：计算两个集合中样本的特征均值的距离

使用最大均值差异度量源域和目标域的距离 $f^* = \arg \min_f \frac{1}{N_s} \sum_{i=1}^{N_s} \ell(f(x_i), y_i) + \lambda D(D_s, D_t)$

● 源域和目标域的距离，可分解为边缘分布和条件分布的最大均值差异：

$$\begin{aligned} D(D_s, D_t) &\approx (1 - \mu)D(P_s(x), P_t(x)) + \mu D(P_s(y|x), P_t(y|x)) \\ &= (1 - \mu)\text{MMD}(P_s(x), P_t(x)) + \mu \text{MMD}(P_s(y|x), P_t(y|x)) \end{aligned}$$

$P_s(x), P_t(x)$: 源域和目标域中样本的边缘分布, $P_s(y|x), P_t(y|x)$: 源域和目标域中样本的条件分布

● 边缘分布的最大均值差异可表示为：

$$\text{MMD}(P_s(x), P_t(x)) = \left\| \frac{1}{N_s} \sum_{i=1}^{N_s} A^T x_i - \frac{1}{N_t} \sum_{j=1}^{N_t} A^T x_j \right\|_H^2$$

A : 将样本映射为特征的变换矩阵, 是待优化的变量

● 条件分布的最大均值差异可近似表示为：

$$\text{MMD}(P_s(y|x), P_t(y|x)) \approx \sum_{c=1}^C \left\| \frac{1}{N_s^{(c)}} \sum_{x_i \in D_s^{(c)}} A^T x_i - \frac{1}{N_t^{(c)}} \sum_{x_j \in D_t^{(c)}} A^T x_j \right\|_H^2$$

(c) : 标识第 c 个类别的对应变量
目标域的 (c) 信息为估计的伪标签

□ 优化目标的化简

● 再考虑特征映射后的散度最大化，即方差最大： $\max(A^T X)H(A^T X)^T$ $H = I - (1/n)1$

● 最小化源域和目标域的均值差异及散度约束，其优化目标可化简为：

$$\begin{aligned} \min D(D_s, D_t) &\quad \min \frac{\text{tr}(A^T X M X^T A)}{\text{tr}(A^T X H X^T A)} \quad \Rightarrow \quad \min \text{tr}(A^T X M X^T A) + \lambda \|A\|_F^2, \\ \max(A^T X)H(A^T X)^T &\quad s.t. A^T X H X^T A = I. \end{aligned}$$

X : 由源域和目标源域样本拼接成的矩阵

M : 称为MMD矩阵，由边缘和条件MMD矩阵线性组合得到

H : 称为中心矩阵，由单位矩阵和常数矩阵相减得到

● 使用拉格朗日法进行求解上式，求解所得矩阵 A 即用于对齐源域和目标域的特征映射矩阵

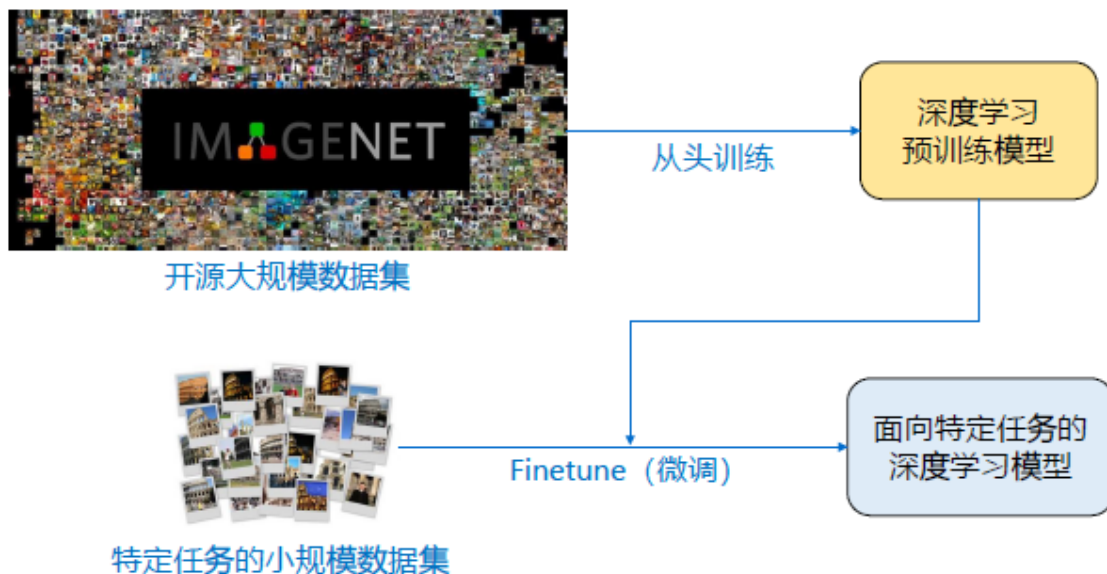
● 以 $(A^T X_s, Y_s)$ 训练分类器 f_i ,再对 $A^T X_t$ 进行测试

深度迁移学习的方式

(掌握概念)

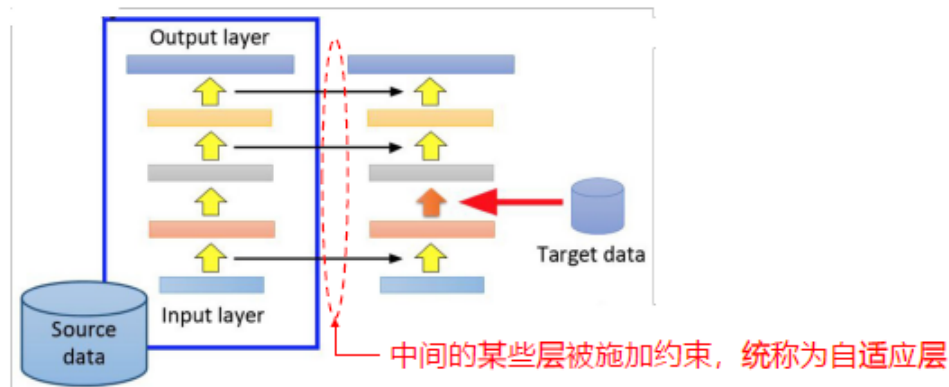
深度网络在前几层关注边缘、纹理等低层信息，后续层关注整体形状、结构等高层信息

□ 第一种方式：基于Finetune（微调）的迁移学习



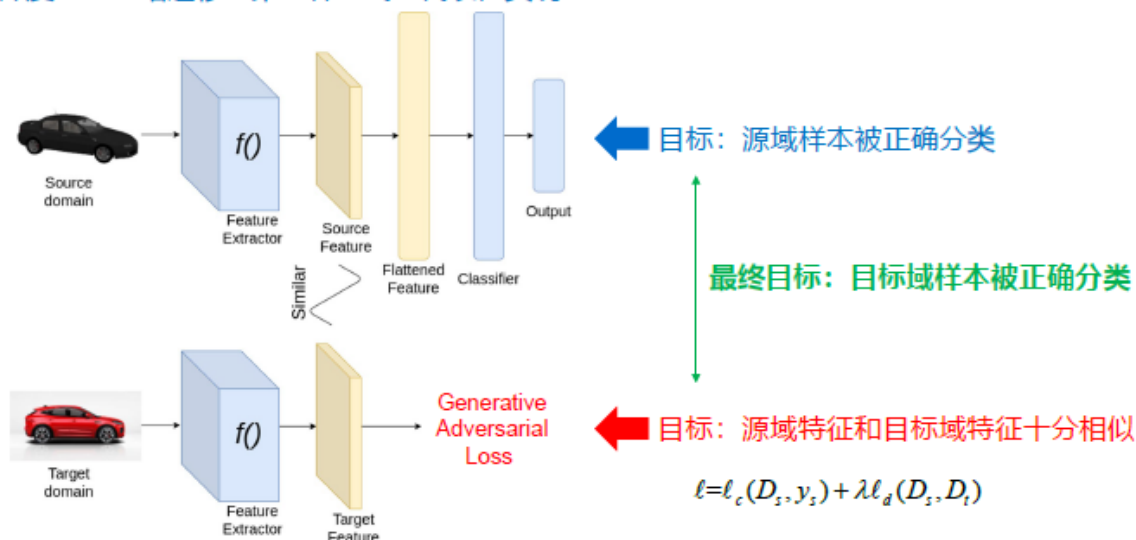
□ 第二种方式：基于自适应层的迁移学习

- 源域和目标域之间差异较大时，此类方法往往优于基于Finetune的方法



核心思想是使得源域和目标域的特征分布更加接近

□ 深度对抗网络迁移：第二种方式的代表性实现

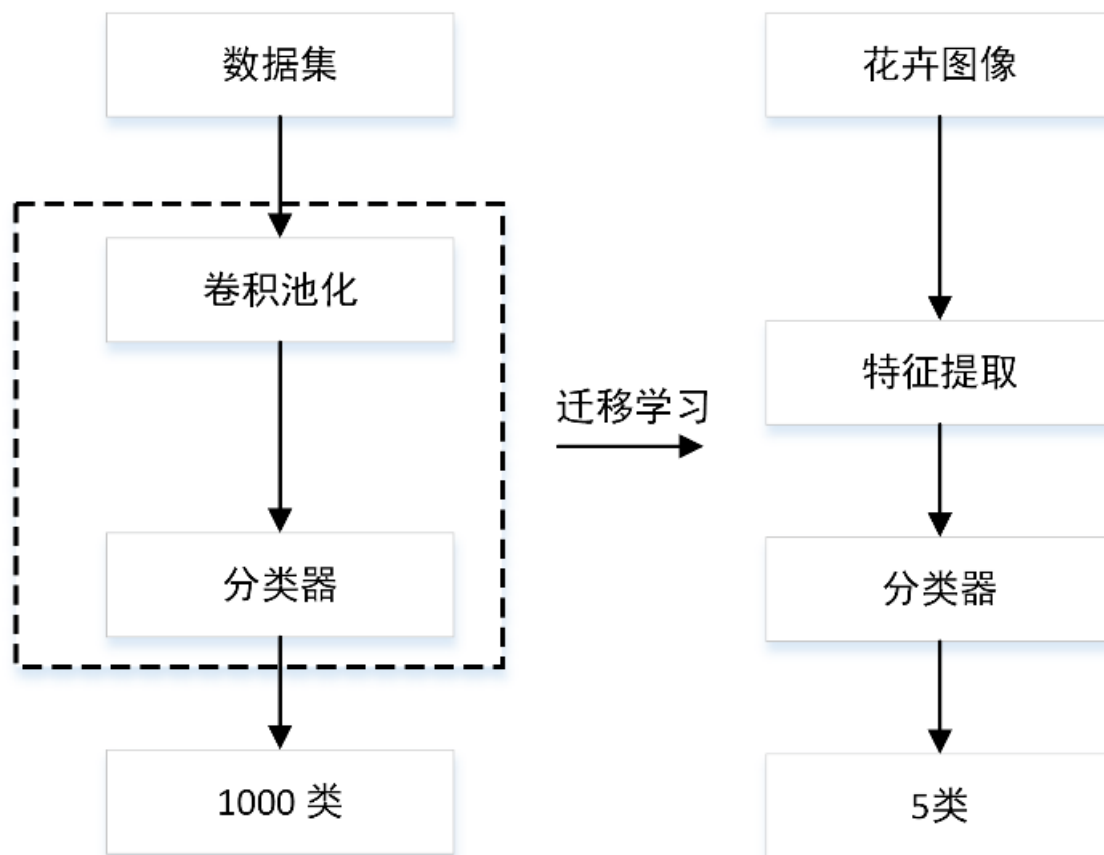


例子中的实现形式

(掌握思路)

花卉种类识别：基于Finetune（微调）的迁移学习

- ImageNet上预训练后，在花卉图像上微调



医学图像分割：基于逐层微调和生成对抗损失的迁移学习

小样本学习

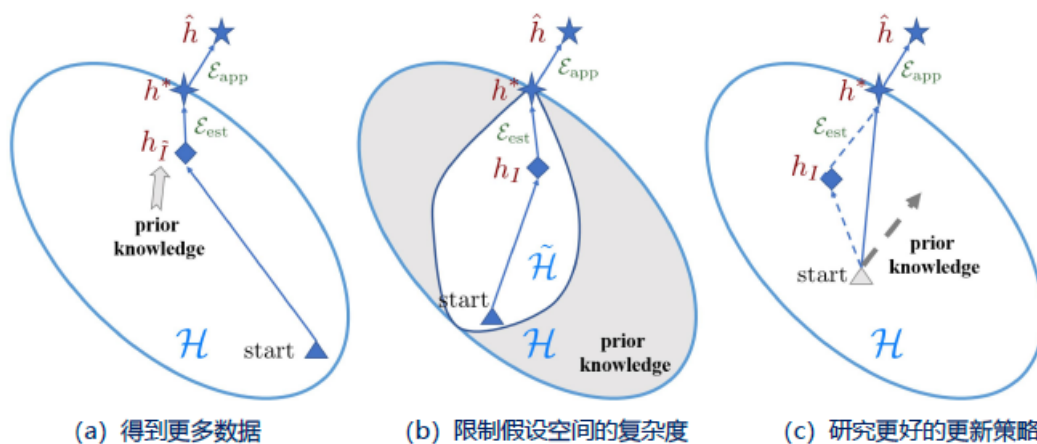
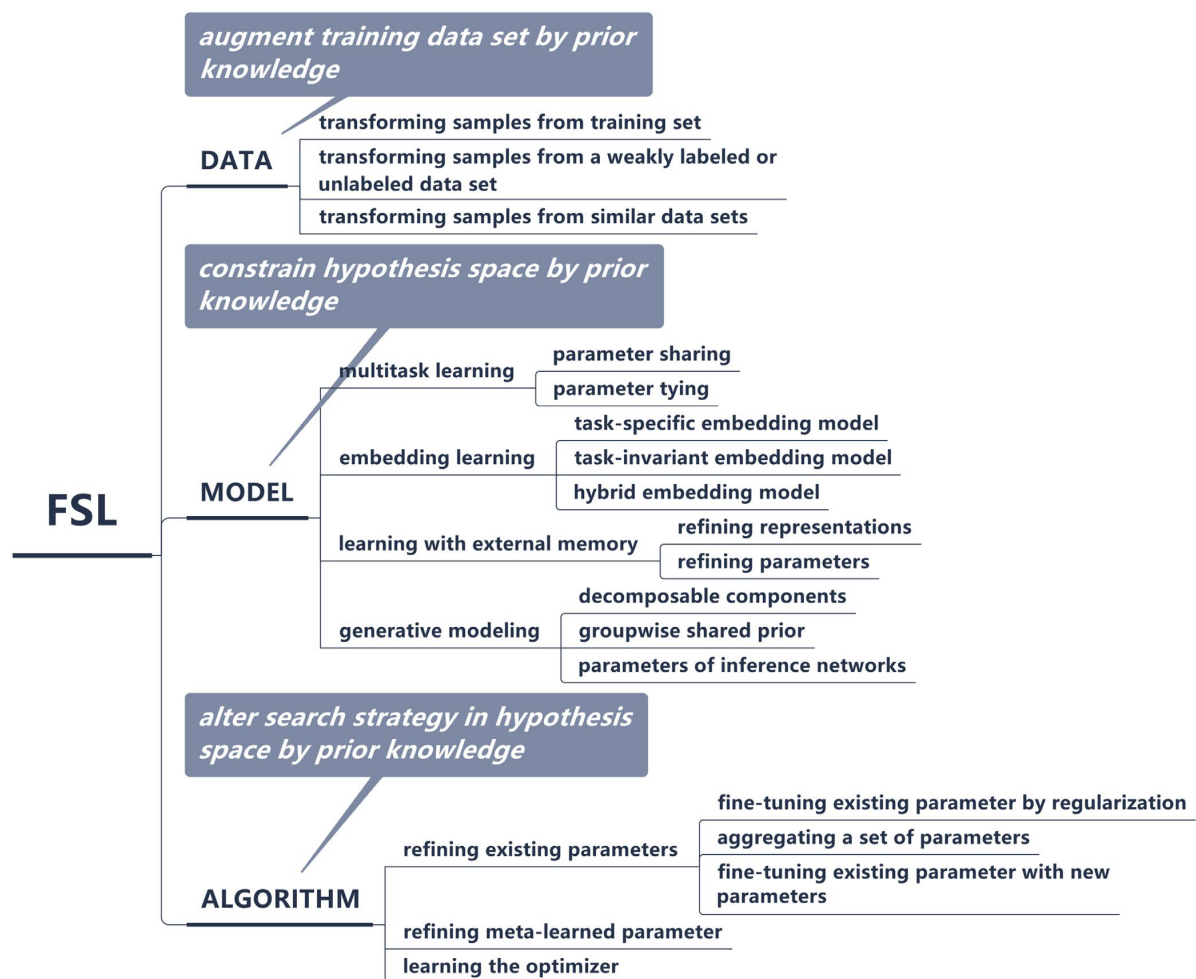
小样本学习问题的定义及算法分类

小样本学习可以视为 N-way、K-shot 的训练问题，即训练集包含N类样本，每类样本包含K个带标签训练数据（K一般小于20）

- 相比于样本充足的情况，小样本条件下学到的最优近似解 h_I 与实际数据可能达到的最优解 h^* 相差更多，即更易过拟合。

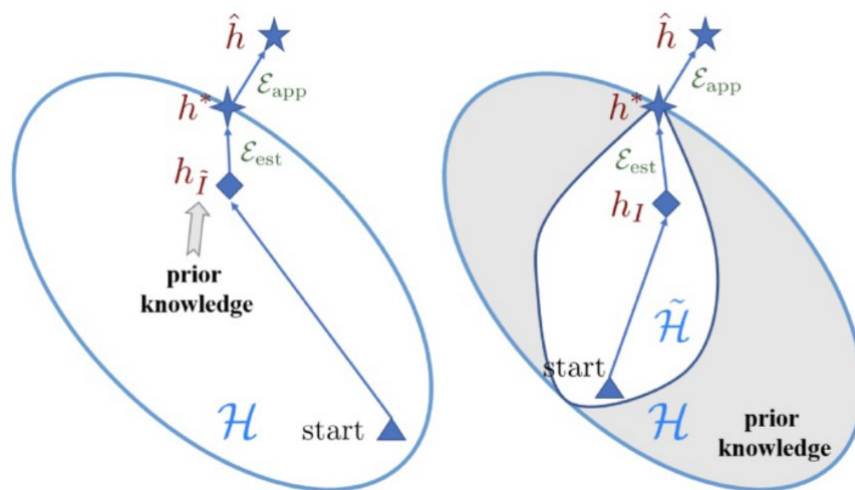
目前小样本学习算法主要分为三大类

- 得到更多数据
- 限制假设空间的复杂度
- 研究更好的学习算法



基于模型优化的基本思想及类型

对于有限样本的训练集，通常会选择较大的假设空间（图a），通过大量训练样本来优化参数，找到最优解。基于模型优化的方法通过先验知识设法约束假设空间来进行学习（图b），降低小样本条件下模型的过拟合风险，提升模型泛化性能。



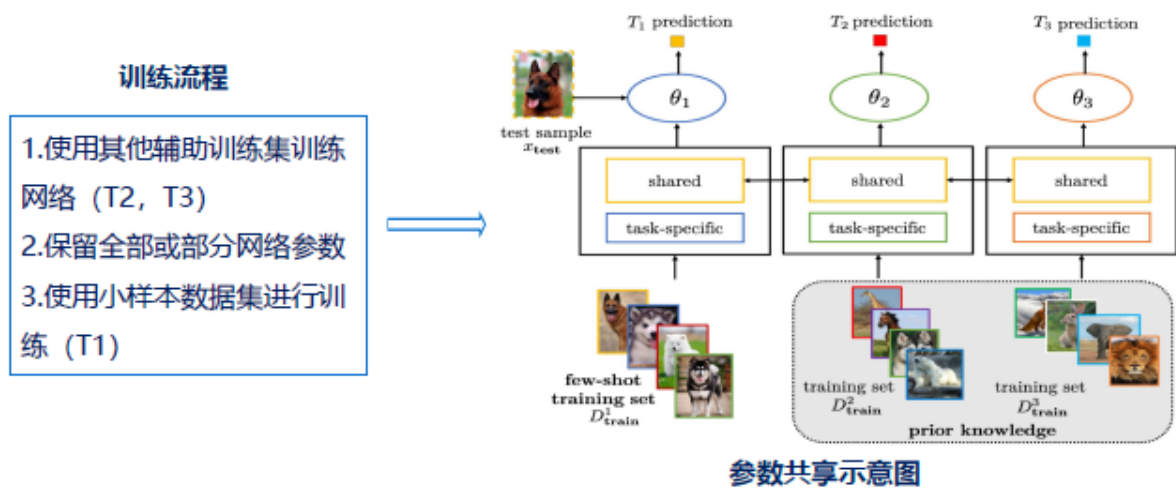
类型

多任务学习	嵌入学习	基于外部记忆	生成建模方法	基于度量学习
在存在多个相关任务的情况下，多任务学习通过利用任务通用信息和任务特定信息来同时学习这些任务。	嵌入的本质是“压缩”，用较低维度的 k 维特征去描述有冗余信息的较高维度的 n 维特征	使用外部记忆学习从训练数据中提取知识，并将其存储在外部存储器中	生成方法是由数据学习联合概率分布 $P(X, Y)$ ，由 $P(Y X) = P(X, Y)/P(X)$ 求出概率分布 $P(Y X)$ 作为预测的模型	学习一种同类样本的特征向量之间距离小，不同类样本的特征向量之间距离大的空间，从而对数据进行区分

多任务学习

给定 C 个相关任务 $T_1, T_1 \dots T_c$ ，其中有的任务属于小样本情况，有的任务含有大量样本。我们本情况对应的任务看做目标任务（target tasks），其余的看做为源任务（source tasks）。所有的任务是同时进行学习的，对于第 c 个任务参数的更新就受限于其他任务的学习，从而目标任务的解空间。根据任务参数的约束方式，该策略中的方法分为参数共享和参数绑定

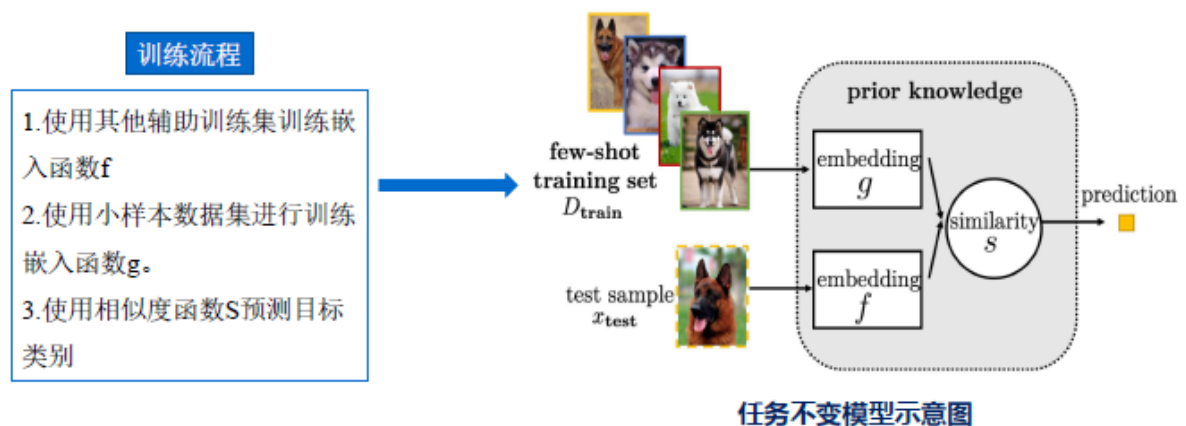
参数共享	参数绑定
此策略在任务之间直接共享一些参数	此策略期望不同任务的参数尽可能的相似



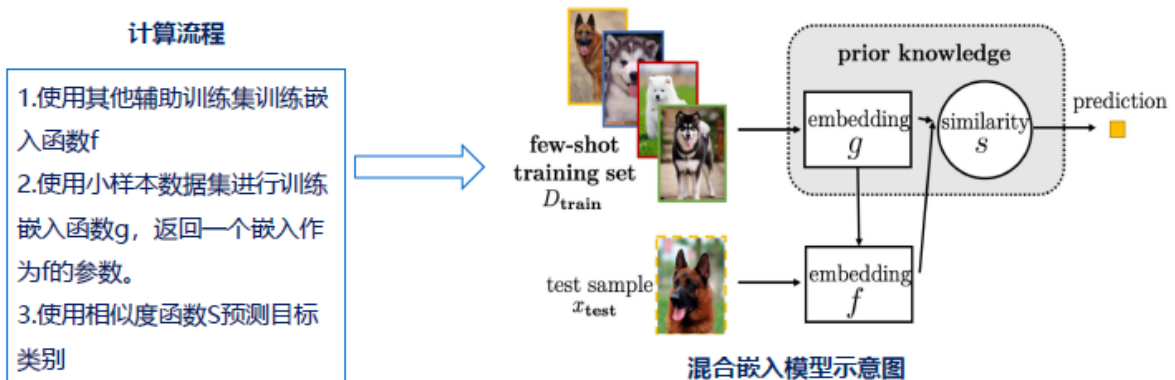
嵌入学习

嵌入 (Embedding) 的本质是“压缩”，用较低维度的 k 维特征去描述有冗余信息的较高维度的 n 维特征。嵌入学习将每个样本 $x_i \in X \subseteq R^d$ 嵌入到低维 $z_i \in Z \subseteq R^m (m < d)$ 。然后，在这个较低维的 Z 中，可以构造一个较小的假设空间 H ，从而减少训练样本的需求量。

任务不变的嵌入方法从包含足够样本的大规模数据集中学习一个通用的嵌入函数，无需重新训练将其直接用于新的小样本数据。

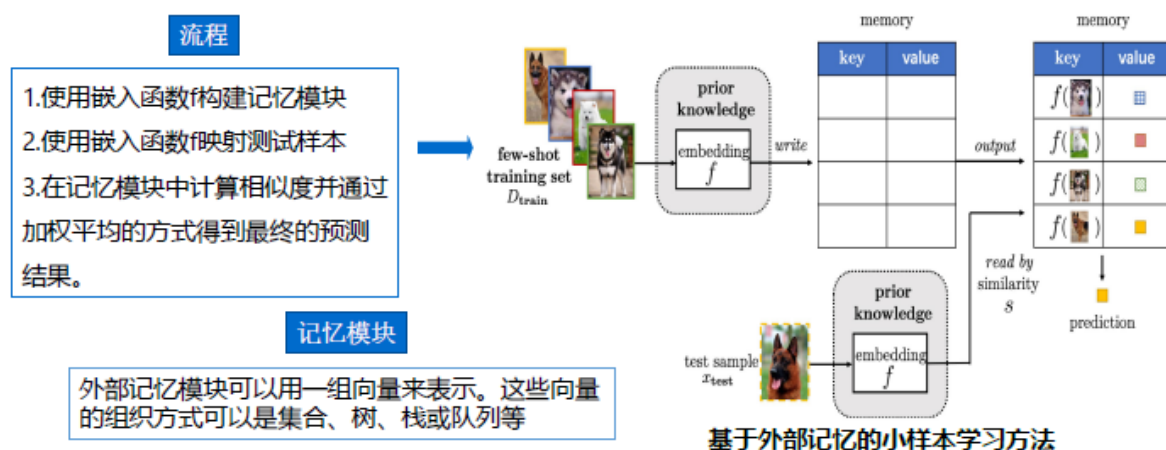


混合嵌入模型采用了一般的任务不变嵌入模型，该模型根据数据训练中的特定任务信息从先验知识中学习一个函数，该函数将从 D_{train} 中提取的信息作为输入并返回一个嵌入到 f 。



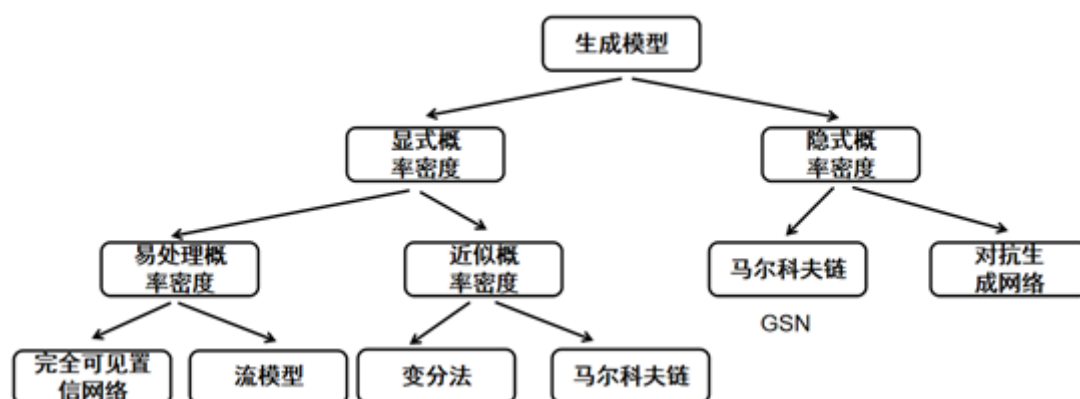
基于外部记忆学习

每个新样本 x_{test} 用从内存中提取内容的加权平均值表示。这限制了 x_{test} 由内存中的内容表示，因此实质上减小了假设空间的大小。这个引入辅助记忆单元一般称为**外部记忆**。



基于生成建模方法

基于训练数据学习联合概率分布 $P(X,Y)$, 然后由 $P_{YX}=P(X,Y)/P(X)$ 求出概率分布 P_{YX} 作为预测的模型, 这就是需要确定的生成模型。该方法目的是表示输入 X 与产生输出 Y 的生成关系



基于度量学习的小样本算法

度量学习是一种空间映射的方法，其能够学习到一种距离度量函数，将所有数据都被转换成一个特征向量，并且相似样本的特征向量之间距离小，不相似样本的特征向量之间距离大，从而对数据进行区分。

元学习

元学习 (Meta-Learning)，又称“学会学习” (Learning to learn)，即利用以往的知识经验来指导新任务的学习，使网络具备学会学习的能力。

- 是要学习一个 F ，用它来学习各种任务的 f
- 通过多个任务的学习，使得模型学习其他 t 任务时更加容易
- 样本数量比较少的任务上，可通过有效率的学习，从而提升准确率和收敛速度

元学习中要准备**许多任务**来进行学习，而每个任务又有**各自的训练集和测试集**，是解决小样本问题（Few-shot Learning）常用的方法之一。

1. 首先在任务1上学习得到模型1，损失 L_1 ；然后在任务2上学习得到模型2，损失 L_2 ；依次计算后，所有的任务损失和即为元学习的损失，令其最小化：

$$L(F) = \sum_{n=1}^N l^n$$

N tasks
Testing loss for task n after training

$$F^* = \arg \min_F L(F)$$

□ 元学习训练流程

1. 准备N个训练任务，每个任务都有对应的Support Set和Query Set。再准备几个测试任务，测试任务用于评估meta learning学习到的参数效果。训练任务和测试任务均从整体数据集中采样得到。
2. 定义网络结构，譬如CNN。并初始化一个meta网络的参数为 ϕ^0 ，meta网络最终要应用到新的测试任务中的网络，该网络中存储了“先验知识”。
3. 开始执行迭代“预训练”：
 - a. **采样一个训练任务m**（或几个训练任务）。将meta网络的参数 ϕ^0 赋值给任务m对应的CNN，得到 θ^m 。
 - b. 使用**任务m的Support Set**，基于**任务m的学习率 α_m** ，对 θ^m 进行**1次更新**，得到 $\hat{\theta}^m$
 - c. 基于1次优化后的 $\hat{\theta}^m$ ，使用**Query Set**计算任务m的loss—— $l^m(\hat{\theta}^m)$ ，并计算其对 $\hat{\theta}^m$ 的梯度。
 - d. 用该梯度，乘**meta网络的学习率 α_{meta}** ，更新 ϕ^0 ，得到 ϕ^1 。

□ 元学习训练流程

- e. 继续采样一个任务n，将 ϕ^1 赋值给任务n对应的CNN网络，得到 θ^n ；
 - f. 使用任务n的训练数据对 θ^n 进行优化得到 $\hat{\theta}^n$ ；
 - g. 基于1次优化后的 $\hat{\theta}^n$ ，使用Query Set计算任务n的loss—— $l^n(\hat{\theta}^n)$ ，并计算其对 $\hat{\theta}^n$ 的梯度；
 - h. 用该梯度，乘meta网络的学习率 α_{meta} ，更新 ϕ^1 ，得到 ϕ^2 ；
 - i. 在数据集中重复a-h的过程；
4. 通过3得到的**meta网络可用于测试任务中**。使用测试任务的**Support Set对meta网络进行微调**。
 5. 最终使用测试任务的Query Set评估meta learning 效果。

注意：

1. meta网络与CNN网络结构相同，参数独立；
2. 每个任务的CNN网络训练完即丢弃。

元学习训练的几点说明

- 所有的任务都服从一个分布 $P(T)$ ，因此采样得到的任务具有一定关联或相似性
- 所有任务共用一个模型，且模型算法应该可由梯度下降法求解
- 学习率有两个：内部任务的学习率和外部元网络的学习率
- 每个任务中的参数只更新一次，最大化学习能力准则（学习不经过长时间复习）

- 单个任务的学习能力要综合到总的学习能力（元模型或元网络）